



Department of Computer Science

Lab Manual

CSC222-Operating System Lab

Instructor's Name: Sir Shahzeb Ansari

Student's Name: Ubaid Khan

Roll No.: 69209 Batch: Spring-2024

Semester: 5th Year: 2026

Department: BSSE



FACULTY OF ENGINEERING SCIENCES AND TECHNOLOGY

Department of Computer Science

Lab Manual

CSC222-L-Operating System Lab

Prepared By:

Mr.Usama Amjad

Reviewed / Approved By:

Muhammad Daud Abbasi

Faculty of Engineering Sciences & Technology

Iqra University

CONTENTS

1. Weekly Plan				
Week	Lecture No	Topic Covered	Cr. Hrs	CLO
1	1	Introduction to UNIX/Linux Shell Environment	3	1-2
2	2	Understand and implement Variable, Special Variable and Arrays in Linux Shell	3	1-2
3	3	Understand and implement Operators in Linux Shell.	3	1-2
4	4	Understand and implement Decision making, Loop, and Shell functions in Linux Shell.	3	1-2
5	5	Understand and implement shell substitution, Quoting mechanisms, I/O redirection and Man_page Help in Linux.	3	1-2
6	6	To understand UNIX/Linux file management by creating, reading, editing and writing in file.	3	1-2
7	7	Open Ended Lab	3	1-2
8	Midterm Exam			
9	9	CPU Scheduling Algorithms: Simulation of FCFS, SJF, RR, Priority scheduling and multi-level queue algorithms, Calculation of Wait time, Turnaround time, Average Wait Time and Average Turnaround time.	3	1-2
10	10	Process Synchronization: Understanding and Simulating Producer Consumer Problem and Dining Philosopher Problem.	3	1-2
11	11	Deadlock Management Techniques: Understanding and Implementing Banker's algorithm,	3	1-2

FACULTY OF ENGINEERING SCIENCES AND TECHNOLOGY

12	12	Memory Management Techniques: Understanding the difference between MVT and MFT. Implementing MFT, MVT, Best fit, Worst fit and First fit Memory Management Techniques.	3	1-2
13	13	Memory Management Techniques using Paging: Understanding the difference between physical address and logical address, Resolving page table. Page Replacement Algorithms: Understanding and Implementing Page Replacement Algorithms(FIFO, LRU, LFU and OPT)	3	1-2
14	14	File Allocation Strategies: Understanding the difference between Indexed, Sequential and Linked File Allocation by implementing them.	3	1-2
15	15	Introduction to kernel development	3	1-2
16	16	Open Ended Lab	3	1-2
17	Final Exam			

Affective Domain Rubrics Assessment

Course Name (Course Code): _____

Semester: _____

CATEGORY	Excellent (100% - 85%)	Good (84% - 75%)	Fair (74% - 60%)	Poor (Less than 60%)
Speaks Clearly	Speaks clearly and distinctly all the time, and confidently.	Speaks clearly and distinctly most of the time, but is confused for a brief period of time, however, recovers.	Speaks clearly and distinctly most of the time, but seems not confident about what has been delivered. Shows lack of confidence.	Often mumbles or cannot be understood and clearly lacks confidence in delivering the content
Points:				
Preparedness	Student is completely prepared and has obviously rehearsed.	Student seems pretty prepared but might have needed a couple more rehearsals.	The student is somewhat prepared, but it is clear that rehearsal was lacking.	Student does not seem at all prepared to present.
Points				
Answer back	Student calmly listens to the questions and responds to the question confidently and correctly	Student calmly listens to the questions, responds confidently but some of the responses are incorrect.	Students show anxiety while listening to the questions, and give some correct responses, but some of the responses are incorrect.	Students show anxiety while listening to the questions, and most of the responses are incorrect.
Points:				
Posture, Eye Contact & Speaking Volume	Stands up straight, looks relaxed and confident. Establishes eye contact with everyone in the room during the presentation. Volume is loud enough to be heard by all members in the audience throughout the presentation.	Stands up straight and establishes eye contact with everyone in the room during the presentation. Volume is loud enough to be heard by the audience, but is sometimes not audible.	Sometimes stands up straight and establishes eye contact. Volume is loud enough to be heard by the audience, but many sentences spoken are not clear.	Lazy and informal posture. Does not look at people during the presentation. Volume is also too soft to be heard by the audience.
Points:				

Overall Score: _____ **out of 10**

Examined by: _____

(Name and Signature of lab instructor)

Open Ended Lab Assessment Rubrics

Course Name (Course Code): _____

Semester: _____

Criteria and Scales			
Excellent (10-8)	Good (7-5)	Average (4-2)	Poor (1-0)
Criterion 1: Understanding the Problem: How well the problem statement is understood by the student			
Understands the problem clearly and clearly identifies the underlying issues.	Adequately understands the problem and identifies the underlying issues.	Inadequately defines the problem and identifies the underlying issues.	Fails to define the problem adequately and does not identify the underlying issues.
Criterion 2: Research: The amount of research that is used in solving the problem			
Contains all the information needed for solving the problem	Good research, leading to a successful solution	Mediocre research which may or may not lead to an adequate solution	No apparent research
Criterion 3: Class Diagram: The completeness of the class diagram			
Class diagram with complete notations	Class diagram with incomplete notations	Class diagram with improper naming convention and notations	No Class diagram
Criterion 4: Code: How complete and accurate the code is along with the assumptions			
Complete Code according to the class diagram of the given case with clear assumptions	Incomplete Code according to the class diagram of the given case with clear assumptions	Incomplete Code according to the class diagram of the given case with unclear assumptions	Wrong code and naming conventions
Criterion 5: Report: How thorough and well organized is the solution			
All the necessary information clearly organized for easy use in solving the problem	Good information organized well that could lead to a good solution	Mediocre information which may or may not lead to a solution	No report provided

Open Ended Lab Assessment Rubrics

Course Name (Course Code): _____

Semester: _____

Criteria and Scales				
Excellent (10-8)	Good (7-5)	Average (4-2)	Poor (1-0)	Total Marks 15
<u>Criterion 1:</u> Understanding the Problem: How well the problem statement is understood by the student				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
<u>Criterion 2:</u> Research: The amount of research that is used in solving the problem				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
<u>Criterion 3:</u> Class Diagram: The completeness of the class diagram				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
<u>Criterion 4:</u> Code: How complete and accurate the code is along with the assumptions				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
<u>Criterion 5:</u> Report: How thorough and well organized is the solution				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Total				(____/15)

Total marks obtained: _____

Name and Signature of lab instructor: _____

Mid Term Rubrics

Course Name (Course Code):

Semester:

Criteria	Exceeds Expectations ($\geq 90\%$)	Meets Expectations (70%-89%)	Developing (50%-69%)	Unsatisfactory ($< 50\%$)
Performance	Able to present full knowledge of both problem and solution.	Able to present adequate knowledge of both problem and solution	Able to present sufficient knowledge of both problem and solution	No or very less knowledge of both problems and solution
Viva	Able to answer the questions easily and correctly	Able to answer the questions	Able to answer the questions but with mistakes	Unable to answer the questions

Mid Term Rubrics based Assessment

Course Name (Course Code):

Semester, Batch:

Score Allocation	
Performance	_____/20
Viva	_____/5
Total Obtained Score	_____/25

Examined by: _____

(Name and Signature of concerned lab instructor)

Final Term Rubrics

Course Name (Course Code): _____

Semester: _____

Criteria	Exceeds Expectations ($\geq 90\%$)	Meets Expectations (70%-89%)	Developing (50%-69%)	Unsatisfactory ($< 50\%$)
Performance	Able to present full knowledge of both problem and solution.	Able to present adequate knowledge of both problem and solution	Able to present sufficient knowledge of both problem and solution	No or very less knowledge of both problems and solution
Viva	Able to answer the questions easily and correctly	Able to answer the questions	Able to answer the questions but with mistakes	Unable to answer the questions

Final Term Rubrics based Assessment

Course Name (Course Code): _____

Semester, Batch: _____

Score Allocation	
Performance	_____/30
Viva	_____/10
Total Obtained Score	_____/40

Examined by: _____

(Name and Signature of concerned lab instructor)

Final Lab Assessment

Assessment Tool	CLO-1 (75)	CLO-2 (25)
Lab Manual		
Subject Project / Viva		
Lab Exam / Viva		
Score Obtained		

Total Score: _____ out of

Examined by: _____

(Name and Signature of concerned lab instructor)

Lab Session 1

Objective:

The Purpose of this Lab is to get familiar with the UNIX/ LINUX Shell Environment.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Introduction:

This lab aims to familiarize you with the UNIX/Linux shell environment, specifically using Fedora in an Oracle Virtual Box setup. The shell is a powerful interface that allows you to interact with the operating system through command-line inputs, making it essential for efficient system management and development.

Procedure:

1. Setting Up Your Environment

Install Oracle Virtual Box:

- Download the latest version of Oracle Virtual Box from the [official website](#).
- Follow the installation instructions for your operating system (Windows, macOS, or Linux).

Download Fedora:

- Visit the [Fedora website](#) and download the latest version of Fedora Workstation (ISO file).
- Ensure that you choose the appropriate version for your architecture (x86_64 for most modern PCs).

Create a Virtual Machine:

- Open Oracle Virtual Box and click on "New".
- Name your virtual machine (e.g., "Fedora Lab") and select "Linux" as the type and "Fedora" as the version.
- Allocate memory (RAM) for the VM (at least 2 GB recommended).
- Create a virtual hard disk (VDI) with a size of at least 20 GB.
- Choose "Start" and select the downloaded Fedora ISO file to boot from it.

2. Installing Fedora

Follow the on-screen instructions to install Fedora:

- Select "Install to Hard Drive".
- Choose your language and region.
- Set up your installation destination (use the default).

- Configure network settings if necessary.
- Create a user account with a strong password.
- Begin the installation and wait for it to complete.
- Once installed, reboot the VM and remove the ISO from the virtual drive when prompted.

3. Exploring the Shell

Access the Terminal:

- Open the Terminal application from the applications menu.

Basic Shell Commands:

- Use `pwd` to display the current working directory.
- Use `ls` to list files and directories in the current directory.
- Use `cd <directory>` to change to a different directory.
- Use `mkdir <directory>` to create a new directory.
- Use `touch <file>` to create a new file.

4. File Operations

Learn to manipulate files using:

- `cp <source> <destination>` to copy files.
- `mv <source> <destination>` to move or rename files.
- `rm <file>` to remove files.
- `cat <file>` to display the contents of a file.

Task #1

Setup Oracle Virtual Box and create a new virtual machine for Fedora installation.

Solution:

Oracle VirtualBox is a virtualization software that allows multiple operating systems to run on a single machine. A new virtual machine is created by selecting Linux as the operating system type and Fedora as the version. Sufficient system resources, including memory and storage space, are allocated to ensure smooth operation. A virtual hard disk is also created to store the Fedora operating system and user data. This virtual environment provides a safe and isolated platform for learning and experimenting with Linux without affecting the host operating system.

Task #2

Download the latest Fedora ISO image from the official Fedora website.

Solution:

The latest Fedora Workstation ISO image is obtained from the official Fedora website. The ISO file contains all the necessary installation files required for setting up the Fedora operating system. The appropriate version corresponding to the system architecture, such as x86_64, is selected to ensure compatibility with modern computers. The downloaded ISO image is later attached to the virtual machine and used as a bootable installation medium.

Task #3

Install Fedora by booting the VM and following the on-screen installation instructions.

Solution:

The virtual machine is booted using the Fedora ISO image. The installation process begins by selecting the "Install to Hard Drive" option. Language preferences, region settings, storage configuration, and user account details are specified during installation. After completing the installation, the system is rebooted and the ISO image is removed from the virtual drive. Once the installation is completed successfully, the Fedora operating system becomes ready for use, providing access to the UNIX/Linux shell environment and basic command-line utilities.

Lab Session 2**Objective:**

Understand and implement Variable, Special Variable and Arrays in Linux Shell.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Introduction:

In this lab, you will learn about variables and special variables in the Linux shell using Fedora in Oracle Virtual Box. Variables are essential for storing data and manipulating information in shell scripts and command-line operations. Special variables provide valuable system and environment information. Understanding these concepts will enhance your ability to write effective shell scripts and automate tasks.

Procedure:**1. Set Up the Environment:**

- Ensure Oracle Virtual Box is installed on your host machine.
- Launch your Fedora virtual machine.

2. Open the Terminal:

- Access the terminal application in your Fedora environment to begin working with shell commands.

Variable Names

- The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).
- By convention, Unix Shell variables would have their names in UPPERCASE.
- The following examples are the university variable names.

- `_UNIVERSITY`
- `TOKEN_A`
- `VAR_1`
- `VAR_2`

Following are not the examples of University variable names –

- `2_VAR`
- `-VARIABLE`
- `VAR1-VAR2`
- `VAR_A!`

02**Iqra University
Engineering****Department of Software**

The reason you cannot use other characters such as !, *, or - is that these characters have a special meaning for the shell.

Defining Variables

Variables are defined as follows –

```
variable name=variable value
```

For example:

```
NAME="IU_student"
```

Above example defines the variable NAME and assigns it the value "IU_Student". Variables of this type are called scalar variables. A scalar variable can hold only one value at a time.

The shell enables you to store any value you want in a variable. For example –

```
VAR1="IU_Student"  
VAR2=100
```

Accessing Values

To access the value stored in a variable, prefix its name with the dollar sign (\$) –

For example, following script would access the value of defined variable NAME and would print it on STDOUT –

```
#!/bin/sh  
  
NAME="IU_Student"  
echo $NAME
```

This would produce following value –

```
IU_Student
```

Read-only Variables

The shell provides a way to mark variables as read-only by using the readonly command. After a variable is marked read-only, its value cannot be changed.

For example, following script would give error while trying to change the value of NAME –

```
#!/bin/sh  
  
NAME="IU_Student"  
readonly NAME  
NAME="New_student"
```

This would produce following result –

```
/bin/sh: NAME: This variable is read only.
```

Unsetting Variables

Unsetting or deleting a variable tells the shell to remove the variable from the list of variables that it tracks. Once you unset a variable, you would not be able to access stored value in the variable.

Following is the syntax to unset a defined variable using the **unset command** –

```
unset variable_name
```

Above command would unset the value of a defined variable. Here is a simple example –

```
#!/bin/sh

NAME="IU_Student"
unset NAME
echo $NAME
```

Above example would not print anything. You cannot use the unset command to **unset** variables that are marked **readonly**.

Variable Types

When a shell is running, three main types of variables are present –

- **Local Variables** – A local variable is a variable that is present within the current instance of the shell. It is not available to programs that are started by the shell. They are set at command prompt.
- **Environment Variables** – An environment variable is a variable that is available to any child process of the shell. Some programs need environment variables in order to function correctly. Usually a shell script defines only those environment variables that are needed by the programs that it runs.
- **Shell Variables** – A shell variable is a special variable that is set by the shell and is required by the shell in order to function correctly. Some of these variables are environment variables whereas others are local variables.

- The following table shows a number of special variables that you can use in your shell scripts

Variable	Description
----------	-------------

\$0	The filename of the current script.
\$n	These variables correspond to the arguments with which a script was invoked. Here n is a positive decimal number corresponding to the position of an argument (the first argument is \$1, the second argument is \$2, and so on).
\$#	The number of arguments supplied to a script.
\$*	All the arguments are double quoted. If a script receives two arguments, \$* is equivalent to \$1 \$2.
\$@	All the arguments are individually double quoted. If a script receives two arguments, @\$ is equivalent to \$1 \$2.
\$?	The exit status of the last command executed.
\$\$	The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
#!	The process number of the last background command.

Command-Line Arguments

The command-line arguments \$1, \$2, \$3,...\$9 are positional parameters, with \$0 pointing to the actual command, program, shell script, or function and \$1, \$2, \$3, ...\$9 as the arguments to the command.

Following script uses various special variables related to command line –

```
#!/bin/sh

echo "File Name: $0"
echo "First Parameter : $1"
echo "Second Parameter : $2"
echo "Quoted Values: @$"
echo "Quoted Values: $*"
echo "Total Number of Parameters : $#"
```

Here is a sample run for the above script –

```
$/test.sh IU Student
File Name : ./test.sh
First Parameter : IU
Second Parameter : Student
Quoted Values: IU Student
Quoted Values: IU Student
Total Number of Parameters : 2
```

02**Iqra University
Engineering****Department of Software****Special Parameters \$* and \$@**

There are special parameters that allow accessing all of the command-line arguments at once. \$* and \$@ both will act the same unless they are enclosed in double quotes, "".

Both the parameter specifies all command-line arguments but the "\$*" special parameter takes the entire list as one argument with spaces between and the "\$@" special parameter takes the entire list and separates it into separate arguments.

We can write the shell script shown below to process an unknown number of command-line arguments with either the \$* or \$@ special parameters –

```
#!/bin/sh

for TOKEN in $*
do
    echo $TOKEN
done
```

There is one sample run for the above script –

```
./test.sh IU Student 17 Years Old
IU
Student
17
Years
Old
```

Exit Status

The \$? variable represents the exit status of the previous command.

Exit status is a numerical value returned by every command upon its completion. As a rule, most commands return an exit status of 0 if they were successful, and 1 if they were unsuccessful.

Some commands return additional exit statuses for particular reasons. For example, some commands differentiate between kinds of errors and will return various exit values depending on the specific type of failure.

Following is the example of successful command –

```
./test.sh IU Student
File Name : ./test.sh
First Parameter : IU
Second Parameter : Student
Quoted Values: IU Student
Quoted Values: IU Student
Total Number of Parameters : 2
$echo $?
0
$
```

Arrays in UNIX/Linux Shell

This lab focuses on understanding array variables in the UNIX/Linux shell. Unlike scalar variables that hold a single value, arrays can store multiple values, facilitating better organization and manipulation of data.

Learn to work with arrays and operators in the Linux shell environment. Shell variables generally store a single value, known as scalar variables. However, shell scripting also supports array variables, which allow storing multiple values within a single variable. Arrays enable efficient grouping of variables, letting you use one array variable to access multiple values, instead of naming each individually. Naming rules for shell variables also apply to arrays. This lab will focus on understanding, creating, and manipulating arrays and operators in Linux.

Defining Array Values

The difference between an array variable and a scalar variable can be explained as follows.

Say that you are trying to represent the names of various students as a set of variables. Each of the individual variables is a scalar variable as follows –

```
NAME01="Iqra"  
NAME02="University"  
NAME03="Computer Lab"  
NAME04="Engineering Lab"  
NAME05="Telecom Lab"
```

We can use a single array to store all the above mentioned names. Following is the simplest method of creating an array variable is to assign a value to one of its indices. This is expressed as follows –

```
array_name[index]=value
```

Here *array_name* is the name of the array, *index* is the index of the item in the array that you want to set, and *value* is the value you want to set for that item.

As an example, the following commands –

```
NAME[0]="Iqra"  
NAME[1]="University"  
NAME[2]="Computer Lab"  
NAME[3]="Engineering Lab"
```

If you are using **ksh** shell the here is the syntax of array initialization –

```
set -A array_name value1 value2 ... valuen
```

If you are using **bash** shell the here is the syntax of array initialization –

```
array_name=(value1 ... valuen)
```

Accessing Array Values

After you have set any array variable, you access it as follows –

```
${array_name[index]}
```

Here *array_name* is the name of the array, and *index* is the index of the value to be accessed. Following is the simplest example –

```
#!/bin/sh
NAME[0]="Iqra"
NAME[1]="University"
NAME[2]="Computer Lab"
NAME[3]="Engineering Lab"
echo "First Index: ${NAME[0]}"
echo "Second Index: ${NAME[1]}"
```

This would produce following result –

```
./test.sh
First Index: Iqra
Second Index: University
```

You can access all the items in an array in one of the following ways –

```
${array_name[*]}
${array_name[@]}
```

Here *array_name* is the name of the array you are interested in. Following is the simplest example –

```
#!/bin/sh

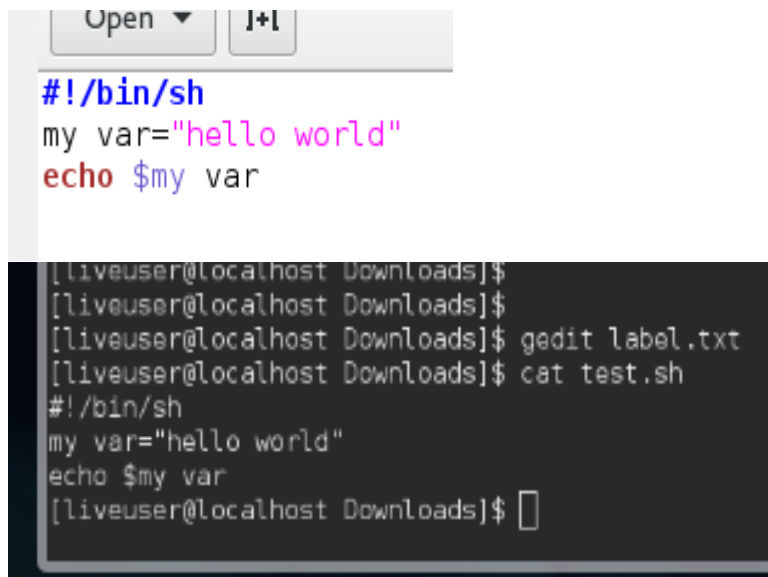
NAME[0]="Iqra"
NAME[1]="University"
NAME[2]="Computer Lab"
NAME[3]="Engineering Lab"
echo "First Method: ${NAME[*]}"
echo "Second Method: ${NAME[@]}"
```

This would produce following result –

```
./test.sh
First Method: Iqra University Computer Lab Engineering Lab
Second Method: Iqra University Computer Lab Engineering Lab
```

Task #1

Create a variable in the shell using `my_var="Hello, World!"` and display it with `echo $my_var`.

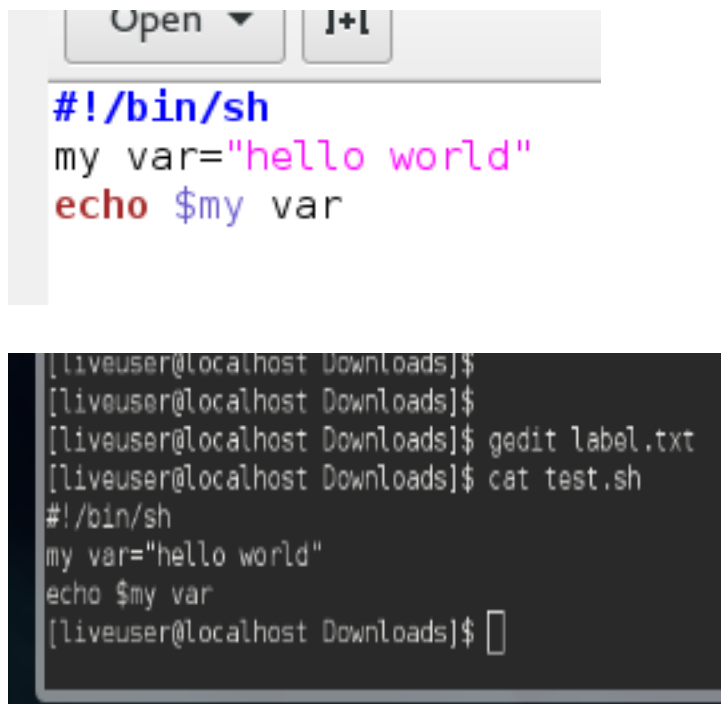


```
Open ▼  I+I
#!/bin/sh
my var="hello world"
echo $my var

[liveuser@localhost Downloads]$
[liveuser@localhost Downloads]$
[liveuser@localhost Downloads]$ gedit label.txt
[liveuser@localhost Downloads]$ cat test.sh
#!/bin/sh
my var="hello world"
echo $my var
[liveuser@localhost Downloads]$
```

Task #2

Create a variable in the shell using `my\_var="Hello, World!"` and display it with `echo \$my\_var`



```
Open ▼  I+I
#!/bin/sh
my var="hello world"
echo $my var

[liveuser@localhost Downloads]$
[liveuser@localhost Downloads]$
[liveuser@localhost Downloads]$ gedit label.txt
[liveuser@localhost Downloads]$ cat test.sh
#!/bin/sh
my var="hello world"
echo $my var
[liveuser@localhost Downloads]$
```

Task #3

Use the read command to get user input and assign it to a variable, then display a greeting.

```
[liveuser@localhost Downloads]$ ./dog.sh
enter your name:izhaan
hello, izhaan! nice to meet you..
[liveuser@localhost Downloads]$
```



Task #4

If you have an array `ANIMALS=("Dog" "Cat" "Bird")`, how would you access and print the second element of this array in a shell script?



```
[liveuser@localhost Downloads]$ gedit asad.sh
** (gedit:3235): WARNING **: Error when getting information
wnloads/asad.sh': No such file or directory
[liveuser@localhost Downloads]$ chmod +x asad.sh
[liveuser@localhost Downloads]$ ./asad.sh
cat
\[liveuser@localhost Downloads]$
```

Lab Session 3

Objective:

Understand and implement Operator in Linux Shell.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Operator

There are various operators supported by each shell. Our tutorial is based on default shell (Bourne) so we are going to cover all the important Bourne Shell operators in the tutorial.

There are following operators which we are going to discuss –

- Arithmetic Operators.
- Relational Operators.
- Boolean Operators.
- String Operators.
- File Test Operators.

The Bourne shell didn't originally have any mechanism to perform simple arithmetic but it uses external programs, either **awk** or the must simpler program **expr**.

Here is simple example to add two numbers –

```
#!/bin/sh
val=`expr 2 + 2`
echo "Total value : $val"
```

This would produce following result –

```
Total value : 4
```

There are following points to note down –

- There must be spaces between operators and expressions, for example 2+2 is not correct, whereas it should be written as 2 + 2.
- Complete expression should be enclosed between `` , called inverted commas.

Arithmetic Operators

There are following arithmetic operators supported by Bourne Shell.

Assume variable a holds 10 and variable b holds 20 then –

Operator	Description	Example
+	Addition - Adds values on either side of the operator	`expr \$a + \$b` will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10
*	Multiplication - Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/	Division - Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
=	Assignment - Assign right operand in left operand	a=\$b would assign value of b into a
==	Equality - Compares two numbers, if both are same then returns true.	[\$a == \$b] would return false.
!=	Not Equality - Compares two numbers, if both are different then returns true.	[\$a != \$b] would return true.

It is very important to note here that all the conditional expressions would be put inside square braces with one space around them, for example [\$a == \$b] is correct whereas [\$a==\$b] is incorrect.

All the arithmetical calculations are done using long integers.

Relational Operators:

Bourne Shell supports following relational operators which are specific to numeric values. These operators would not work for string values unless their value is numeric.

For example, following operators would work to check a relation between 10 and 20 as well as in between "10" and "20" but not in between "ten" and "twenty".

Assume variable a holds 10 and variable b holds 20 then –

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not, if yes then condition becomes true.	[\$a -eq \$b] is not true.
-ne	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	[\$a -ne \$b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	[\$a -gt \$b] is not true.

-lt	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	[\$a -lt \$b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	[\$a -ge \$b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	[\$a -le \$b] is true.

It is very important to note here that all the conditional expressions would be put inside square braces with one space around them, for example [\$a <= \$b] is correct whereas [\$a <= \$b] is incorrect.

Boolean Operators

There are following boolean operators supported by Bourne Shell.

Assume variable a holds 10 and variable b holds 20 then –

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical OR. If one of the operands is true then condition would be true.	[\$a -lt 20 -o \$b -gt 100] is true.
-a	This is logical AND. If both the operands are true then condition would be true otherwise it would be false.	[\$a -lt 20 -a \$b -gt 100] is false.

String Operators

There are following string operators supported by Bourne Shell.

Assume variable a holds "abc" and variable b holds "efg" then –

Operator	Description	Example
=	Checks if the value of two operands are equal or not, if yes then condition becomes true.	[\$a = \$b] is not true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	[\$a != \$b] is true.

-z	Checks if the given string operand size is zero. If it is zero length then it returns true.	[-z \$a] is not true.
-n	Checks if the given string operand size is non-zero. If it is non-zero length then it returns true.	[-n \$a] is not false.
str	Check if str is not the empty string. If it is empty then it returns false.	[\$a] is not false.

Task #1

Use a bash script to add and subtract two numbers given by the user. Display the results of both operations.

```
[liveuser@localhost ~]$ chmod +x hello.sh
[liveuser@localhost ~]$ ./hello.sh
enter num1
20
enter num2
22
addition of 20 and 22 is 42
[liveuser@localhost ~]$ gedit hello.sh
[liveuser@localhost ~]$ chmod +x hello.sh
[liveuser@localhost ~]$ ./hello.sh
enter num1
24
enter num2
12
subtraction of 24 and 12 is 12
[liveuser@localhost ~]$
```

Task #2

Use a shell script to multiply and divide two numbers entered by the user, displaying the results of both operations.

```
[liveuser@localhost ~]$ gedit hello.sh
[liveuser@localhost ~]$ chmod +x hello.sh
[liveuser@localhost ~]$ ./hello.sh
enter num1
25
enter num2
20
mutiplication of 25 and 20 is 500
[liveuser@localhost ~]$
```

```
[liveuser@localhost ~]$ gedit hello.sh
[liveuser@localhost ~]$ chmod +x hello.sh
[liveuser@localhost ~]$ ./hello.sh
enter num1
13
enter num2
15
division of 13 and 15 is 0
[liveuser@localhost ~]$
```

Task #3

Use a shell script to check if a number entered by the user is even or odd.

```
[liveuser@localhost ~]$ gedit hello.sh
[liveuser@localhost ~]$ chmod +x hello.sh
[liveuser@localhost ~]$ ./hello.sh
enter num
24
24 is even
```

```
#!/bin/sh
echo "enter num"
read num

if(( num % 2 == 0 ));
then
echo "$num is even"
else "$num is odd"
fi
```

```
[liveuser@localhost ~]$ chmod +x hello
[liveuser@localhost ~]$ ./hello.sh
enter num
11
./hello.sh: line 8: 11 is odd: command
[liveuser@localhost ~]$ gedit hello.sh
[liveuser@localhost ~]$
```

```
#!/bin/sh
echo "enter num"
read num

if(( num % 2 == 0 ));
then
echo "$num is even"
else "$num is odd"
fi
```

Lab Session 4

Objective:

Understand and implement Decision making, Loop, and Shell functions in Linux Shell.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Introduction

Linux Shell supports conditional statements which are used to perform different actions based on different conditions. Here we will explain following two decision making statements –

- The **if...else** statements
- The **case...esac** statement

The if...else statements:

If else statements are useful decision making statements which can be used to select an option from a given set of options.

Unix Shell supports following forms of if..else statement –

if...fi statement:

The **if...fi** statement is the fundamental control statement that allows Shell to make decisions and execute statements conditionally.

Syntax

```
if [ expression ]  
then  
    Statement(s) to be executed if expression is true  
fi
```

Here Shell *expression* is evaluated. If the resulting value is *true*, given *statement(s)* are executed. If *expression* is *false* then no statement would be not executed. Most of the times you will use comparison operators while making decisions.

Give you attention on the spaces between braces and expression. This space is mandatory otherwise you would get syntax error.

If **expression** is a shell command then it would be assumed true if it return 0 after its execution. If it is a boolean expression then it would be true if it returns true.

Example

This will produce following result –

```
#!/bin/sh
a=10
b=20
if [ $a == $b ]
then
    echo "a is equal to b"
fi
if [ $a != $b ]
then
    echo "a is not equal to b"
fi
```

```
a is not equal to b
```

if...else...fi statement:

The **if...else...fi** statement is the next form of control statement that allows Shell to execute statements in more controlled way and making decision between two choices.

Syntax

```
if [ expression ]
then
    Statement(s) to be executed if expression is true
else
    Statement(s) to be executed if expression is not true
fi
```

Here Shell *expression* is evaluated. If the resulting value is *true*, given *statement(s)* are executed. If *expression* is *false* then no statement would be not executed.

Example

If we take above example then it can be written in better way using *if...else* statement as follows –

```
#!/bin/sh
```

```
a=10
```

```
b=20
```

```
if [ $a == $b ]
```

```
then
```

```
    echo "a is equal to b"
```

```
else
```

```
    echo "a is not equal to b"
```

```
fi
```

This will produce following result –

```
a is not equal to b
```

if...elif...else...fi statement:

The **if...elif...fi** statement is the one level advance form of control statement that allows Shell to make correct decision out of several conditions.

Syntax

```
if [ expression 1 ]
```

```
then
```

```
    Statement(s) to be executed if expression 1 is true
```

```
elif [ expression 2 ]
```

```
then
```

```
    Statement(s) to be executed if expression 2 is true
```

```
elif [ expression 3 ]
```

```
then
```

```
    Statement(s) to be executed if expression 3 is true
```

```
else
```

```
    Statement(s) to be executed if no expression is true
```

```
fi
```

There is nothing special about this code. It is just a series of *if* statements, where each *if* is part of the *else* clause of the previous statement. Here statement(s) are executed based on the true condition, if none of the condition is true then *else* block is executed.

Example

```
#!/bin/sh
a=10
b=20
if [ $a == $b ]
then
    echo "a is equal to b"
elif [ $a -gt $b ]
then
    echo "a is greater than b"
elif [ $a -lt $b ]
then
    echo "a is less than b"
else
    echo "None of the condition met"
fi
```

This will produce following result –

```
a is less than b
```

Loops:

Loops are a powerful programming tool that enable you to execute a set of commands repeatedly. In this tutorial, you would examine the following types of loops available to shell programmers –

The while loop

The while loop enables you to execute a set of commands repeatedly until some condition occurs. It is usually used when you need to manipulate the value of a variable repeatedly.

Syntax

```
while command
do
    Statement(s) to be executed if command is true
done
```

Here Shell *command* is evaluated. If the resulting value is *true*, given *statement(s)* are executed. If *command* is *false* then no statement would be not executed and program would jump to the next line after done statement.

Example

Here is a simple example that uses the while loop to display the numbers zero to nine –

This will produce following result –

The for loop

The for loop operate on lists of items. It repeats a set of commands for every item in a list.

```
#!/bin/sh
a=0
while [ $a -lt 10 ]
do
    echo $a
    a=`expr $a + 1`
done
```

This will produce following result –

```
0 1 2 3 4 5 6 7 8 9
```

Syntax

```
for var in word1 word2 ... wordN
do
    Statement(s) to be executed for every word.
done
```

Here *var* is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words). Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

Example

Here is a simple example that uses for loop to span through the given list of numbers –

```
#!/bin/sh
for var in 0 1 2 3 4 5 6 7 8 9
do
    echo $var
```

```
done
```

This will produce following result –

```
0 1 2 3 4 5 6 7 8 9
```

The until loop

The while loop is perfect for a situation where you need to execute a set of commands while some condition is true. Sometimes you need to execute a set of commands until a condition is true.

Syntax

```
until command
do
    Statement(s) to be executed until command is true
done
```

Here Shell *command* is evaluated. If the resulting value is *false*, given *statement(s)* are executed. If *command* is *true* then no statement would be not executed and program would jump to the next line after done statement.

Example

Here is a simple example that uses the until loop to display the numbers zero to nine –

This will produce following result –

```
#!/bin/sh

a=0

until [ ! $a -lt 10 ]
do
    echo $a
    a=`expr $a + 1`
done
```

```
0 1 2 3 4 5 6 7 8 9
```

The select loop

You would use different loops based on different situation. For example while loop would execute given commands until given condition remains true whereas until loop would execute until a given condition becomes true.

The *select* loop provides an easy way to create a numbered menu from which users can select options. It is useful when you need to ask the user to choose one or more items from a list of choices.

Syntax

```
select var in word1 word2 ... wordN
do
    Statement(s) to be executed for every word.
done
```

Here *var* is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words). Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

For every selection a set of commands would be executed with-in the loop. This loop was introduced in ksh and has been adapted into bash. It is not available in sh.

Example

Here is a simple example to let the user select a drink of choice –

```
#!/bin/ksh
select DRINK in tea cofee water juice appe all none
do
    case $DRINK in
        tea|cofee|water|all)
            echo "Go to canteen"
            ;;
        juice|appe)
            echo "Available at home"
            ;;
        none)
            break
        *)
            echo "ERROR: InvUniversityd selection"
            ;;
    esac
done
```

The menu presented by the select loop looks like the following –

```
$/test.sh
1) tea
2) cofee
3) water
4) juice
5) appe
6) all
7) none
#? juice
Available at home
#? none
```

\$

So far you have looked at creating loops and working with loops to accomplish different tasks. Sometimes you need to stop a loop or skip iterations of the loop.

In this tutorial you will learn following two statements used to control shell loops –

- The **break** statement
- The **continue** statement

The infinite Loop

All the loops have a limited life and they come out once the condition is false or true depending on the loop.

A loop may continue forever due to required condition is not met. A loop that executes forever without terminating executes an infinite number of times. For this reason, such loops are called infinite loops.

Example

Here is a simple example that uses the while loop to display the numbers zero to nine –

```
#!/bin/sh  
  
a=10  
  
until [ $a -lt 10 ]  
do  
    echo $a  
    a=`expr $a + 1`  
done
```

This loop would continue forever because a is always greater than or equal to 10 and it would never become less than 10.

The break statement

The **break** statement is used to terminate the execution of the entire loop, after completing the execution of all of the lines of code up to the break statement. It then steps down to the code following the end of the loop.

Syntax

The following **break** statement would be used to come out of a loop –

```
break
```

The break command can also be used to exit from a nested loop using this format –

```
break n
```

Here **n** specifies the nth enclosing loop to exit from.

Example

Here is a simple example which shows that loop would terminate as soon as a becomes 5:

This will produce following result –

```
#!/bin/sh

a=0

while [ $a -lt 10 ]
do
    echo $a
    if [ $a -eq 5 ]
    then
        break
    fi
    a=`expr $a + 1`
done
```

This will produce following result –

```
0 1 2 3 4 5
```

Here is a simple example of nested for loop. This script breaks out of both loops if var1 equals 2 and var2 equals 0 –

```
#!/bin/sh

for var1 in 1 2 3
do
    for var2 in 0 5
    do
        if [ $var1 -eq 2 -a $var2 -eq 0 ]
        then
            break 2
        else
            echo "$var1 $var2"
        fi
    done
done
```

This will produce following result. In the inner loop, you have a break command with the argument 2. This indicates that if a condition is met you should break out of outer loop and ultimately from inner loop as well.

```
1 0
1 5
```

Functions

Functions enable you to break down the overall functionUniversityty of a script into smaller, logical subsections, which can then be called upon to perform their individual task when it is needed.

Using functions to perform repetitive tasks is an excellent way to create code reuse. Code reuse is an important part of modern object-oriented programming principles.

Shell functions are similar to subroutines, procedures, and functions in other programming languages.

Creating Functions

To declare a function, simply use the following syntax –

```
function_name () {  
    list of commands  
}
```

The name of your function is `function_name`, and that's what you will use to call it from elsewhere in your scripts. The function name must be followed by parentheses, which are followed by a list of commands enclosed within braces.

Example

Following is the simple example of using function –

When you would execute above script it would produce following result –

```
#!/bin/sh  
  
# Define your function here  
Hello () {  
    echo "Hello World"  
}  
  
# Invoke your function  
Hello
```

When you would execute above script it would produce following result –

```
$/test.sh  
Hello World  
$
```

Pass Parameters to a Function

You can define a function which would accept parameters while calling those function. These parameters would be represented by \$1, \$2 and so on.

Following is an example where we pass two parameters *Iqra* and *University* and then we capture and print these parameters in the function.

This would produce following result –

```
#!/bin/sh

# Define your function here
Hello () {
    echo "Hello World $1 $2"
}

# Invoke your function
Hello Iqra University
```

This would produce following result –

```
$/test.sh
Hello World Iqra University
$
```

Returning Values from Functions

If you execute an exit command from inside a function, its effect is not only to terminate execution of the function but also of the shell program that called the function.

If you instead want to just terminate execution of the function, then there is way to come out of a defined function.

Based on the situation you can return any value from your function using the **return** command whose syntax is as follows –

```
return code
```

Here *code* can be anything you choose here, but obviously you should choose something that is meaningful or useful in the context of your script as a whole.

Example

Following function returns a value 1 –

```
#!/bin/sh

# Define your function here
Hello () {
    echo "Hello World $1 $2"
    return 10
}
```

```
# Invoke your function
Hello Iqra University

# Capture value returned by last command
ret=$?

echo "Return value is $ret"
```

This would produce following result –

```
$/test.sh
Hello World Iqra University
Return value is 10
$
```

Nested Functions

One of the more interesting features of functions is that they can call themselves as well as call other functions. A function that calls itself is known as a *recursive function*.

Following simple example demonstrates a nesting of two functions –

```
#!/bin/sh

# Calling one function from another
number_one () {
    echo "This is the first function speaking..."
    number_two
}

number_two () {
    echo "This is now the second function speaking..."
}

# Calling function one.
number_one
```

This would produce following result –

```
This is the first function speaking...
This is now the second function speaking...
```

Task 1:

Write a shell script using if and else to check if a number is positive or negative.

```
#!/bin/sh

echo "enter a number :"
read num

if [ $num -gt 0 ]; then
echo "number is greater than 0 "
elif [ $num -lt 0 ]; then
echo "number is less than 0 "
else
echo "the number is zero"
fi
```

```
[liveuser@localhost Downloads]$ gedit os.sh
[liveuser@localhost Downloads]$ ./os.sh
enter a number :
5
number is greater than 0
[liveuser@localhost Downloads]$
```

Task 2:

Write a shell script using a for loop to print numbers from 1 to 10.

```
#!/bin/sh
count=1
while [ $count -le 5 ]
do
    echo "Number: $count"
    count=`expr $count + 1`
done
```

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
[liveuser@localhost ~]
```

Task 3:

Write a shell script using a while loop to calculate the factorial of a given number.

```
#!/bin/sh
echo "Files in current directory:"
for file in *
do
    echo "$file"
done
```

```
[liveuser@localhost ~]$ ./haha
Files in current directory:
Desktop
Documents
Downloads
haha.sh
hello.sh
izhaan.sh
Music
Pictures
Public
Templates
test.txt
Videos
[liveuser@localhost ~]$ gedit
[liveuser@localhost ~]$
```

Lab Session 5

Objective:

Understand and implement shell substitution, Quoting mechanisms, I/O redirection and Man_page Help in Linux.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Introduction

In the world of Linux, mastering the terminal and understanding the underlying concepts of shell substitution, quoting mechanisms, I/O redirection, and man pages is fundamental for effective system administration and development. These concepts allow users to interact with the system more efficiently, automate tasks, and manage files and processes seamlessly.

This document introduces these key concepts in Linux, specifically within the context of Fedora, a popular Linux distribution. The aim is to provide an understanding of shell commands, how they can be customized and redirected, and how to use the built-in help system effectively.

Shell Substitution: Shell substitution allows the user to modify the behavior of commands by automatically replacing certain parts of a command line. There are various types of substitution, such as command substitution (replacing a command with its output) and variable substitution (replacing a variable with its value). These techniques are essential for creating powerful one-liner commands and scripts.

Example

Command substitution is generally used to assign the output of a command to a variable. Each of the following examples demonstrate command substitution.

```
#!/bin/sh
```

```
DATE=`date`  
echo "Date is $DATE"
```

```
USERS=`who | wc -l`  
echo "Logged in user are $USERS"
```

```
UP=`date ; uptime`
echo "Uptime is $UP"
```

```
Date is Thu Jul 2 03:59:57 MST 2009
Logged in user are 1
Uptime is Thu Jul 2 03:59:57 MST 2009
03:59:57 up 20 days, 14:03, 1 user, load avg: 0.13, 0.07, 0.15
```

Variable Substitution

Variable substitution enables the shell programmer to manipulate the value of a variable based on its state.

Here is the following table for all the possible substitutions –

Form	Description
<code>\${var}</code>	Substitute the value of <i>var</i> .
<code>\${var:-word}</code>	If <i>var</i> is null or unset, <i>word</i> is substituted for var . The value of <i>var</i> does not change.
<code>\${var:=word}</code>	If <i>var</i> is null or unset, <i>var</i> is set to the value of word .
<code>\${var:?message}</code>	If <i>var</i> is null or unset, <i>message</i> is printed to standard error. This checks that variables are set correctly.
<code>\${var:+word}</code>	If <i>var</i> is set, <i>word</i> is substituted for <i>var</i> . The value of <i>var</i> does not change.

Example

Following is the example to show various states of the above substitution –

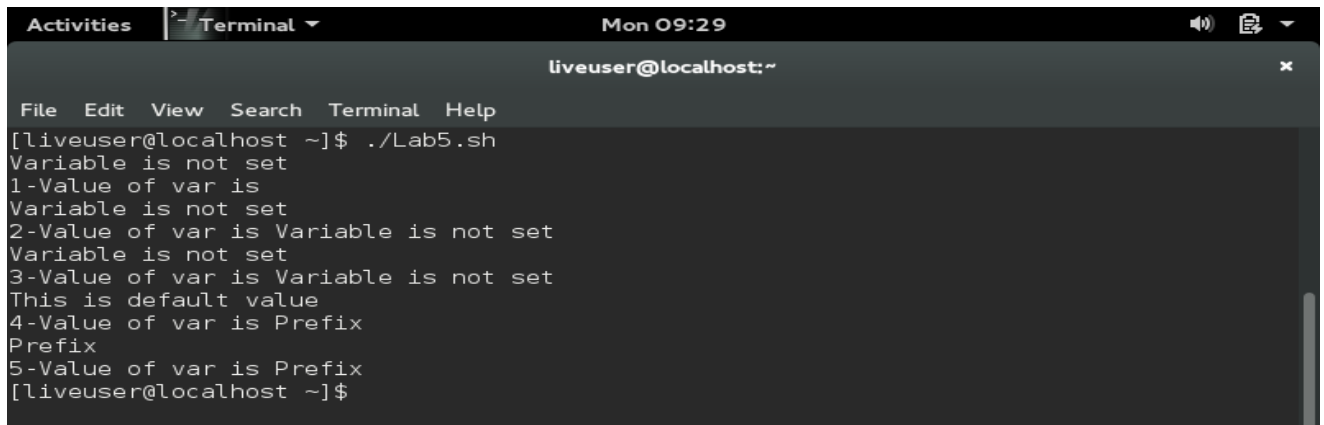
```
#!/bin/sh
```

```
echo ${var:-"Variable is not set"}
```

```
echo "1-Value of var is ${var}"
```

```
echo ${var:="Variable is not set"}
```

```
echo "2-Value of var is ${var}"  
echo ${var:+"Variable is not set"}  
echo "3-Value of var is ${var}"  
var="Prefix"  
echo ${var:+"This is default value"}  
echo "4-Value of var is $var"  
echo ${var:? "Print this Message"}  
echo "5-Value of var is $var"  
  
This would produce following result –
```



The screenshot shows a terminal window titled 'Terminal' with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar (liveuser@localhost:~). The terminal output shows the execution of a script named 'Lab5.sh'. The script prints 'Variable is not set' twice, then '1-Value of var is Variable is not set' and '2-Value of var is Variable is not set'. It then prints '3-Value of var is Variable is not set' and 'This is default value'. Finally, it prints '4-Value of var is Prefix' and '5-Value of var is Prefix'.

```
liveuser@localhost:~  
File Edit View Search Terminal Help  
[liveuser@localhost ~]$ ./Lab5.sh  
Variable is not set  
1-Value of var is  
Variable is not set  
2-Value of var is Variable is not set  
Variable is not set  
3-Value of var is Variable is not set  
This is default value  
4-Value of var is Prefix  
Prefix  
5-Value of var is Prefix  
[liveuser@localhost ~]$
```

Quoting mechanisms:

The Meta characters

Unix Shell provides various meta characters which have special meaning while using them in any Shell Script and causes termination of a word unless quoted.

For example, matches with a single character while listing files in a directory and an * would match more than one characters. Here is a list of most of the shell special characters (also called meta characters)

```
* ? [ ] ' " \ $ ; & ( ) | ^ < > new-line space tab
```

Example

Following is the example which show how to print a * or a ? –

```
#!/bin/sh  
echo Hello; Word
```

this would produce following result –

```
Hello  
./test.sh: line 2: Word: command not found  
  
shell returned 127
```

Now let us try using a quoted character –

```
#!/bin/sh  
  
echo Hello\; Word
```

This would produce following result –

```
Hello; Word
```

The \$ sign is one of the metacharacters, so it must be quoted to avoid special handling by the shell –

```
#!/bin/sh  
  
echo "I have \$1200"
```

```
I have $1200
```

There are following four forms of quoting's –

Quoting	Description
Single quote	All special characters between these quotes lose their special meaning.
Double quote	Most special characters between these quotes lose their special meaning with these exceptions: • \$

	• ` • \\$ • \' • \" • \\
Backslash	Any character immediately following the backslash loses its special meaning.
Back Quote	Anything in between back quotes would be treated as a command and would be executed.

The Single Quotes

Consider an echo command that contains many special shell characters –

```
echo <-$1500.**>; (update?) [y|n]
```

Putting a backslash in front of each special character is tedious and makes the line difficult to read –

```
echo \<-$1500.\*\*>; \ (update\?) \[y\|n\]
```

There is an easy way to quote a large group of characters. Put a single quote (') at the beginning and at the end of the string –

```
echo '<-$1500.**>; (update?) [y|n]'
```

Any characters within single quotes are quoted just as if a backslash is in front of each character. So now this echo command displays properly.

If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (\) as follows –

```
echo 'It\'s Shell Programming'
```

The Double Quotes

Try to execute the following shell script. This shell script makes use of single quote –

```
VAR=IQRA  
echo '$VAR owes <-$1500.**>; [ as of (`date +%m/%d`) ]'
```

This would produce following result –

```
$VAR owes <-$1500.**>; [ as of (`date +%m/%d`) ]
```

So this is not what you wanted to display. It is obvious that single quotes prevent variable substitution. If you want to substitute variable values and to make invert commas work as expected, then you would need to put your commands in double quotes as follows –

```
VAR=IQRA  
echo "$VAR owes <-\$1500.**>; [ as of (`date +%m/%d`) ]"
```

Now this would produce following result –

```
IQRA owes <-$1500.**>; [ as of (07/02) ]
```

Double quotes take away the special meaning of all characters except the following –

- \$ for parameter substitution.
- Backquotes for command substitution.
- \\$ to enable literal dollar signs.
- ` to enable literal backquotes.
- \" to enable embedded double quotes.
- \\ to enable embedded backslashes.
- All other \ characters are literal (not special).

Any characters within single quotes are quoted just as if a backslash is in front of each character. So now this echo command displays properly.

If a single quote appears within a string to be output, you should not put the whole string within single quotes instead you should precede that using a backslash (\) as follows –

```
echo 'It\'s Shell Programming'
```

The Back Quotes

Putting any Shell command in between back quotes would execute the command

Syntax:

Here is the simple syntax to put any Shell **command** in between back quotes –

Example

```
var=`command`
```

Following would execute **date** command and produced result would be stored in DATA variable.

```
DATE=`date`
```

```
echo "Current Date: $DATE"
```

This would produce following result –

```
Current Date: Thu Jul 2 05:28:45 MST 2009
```

I/O Redirection

Most Unix system commands take input from your terminal and send the resulting output back to your terminal. A command normally reads its input from a place called standard input, which happens to be your terminal by default. Similarly, a command normally writes its output to standard output, which is also your terminal by default.

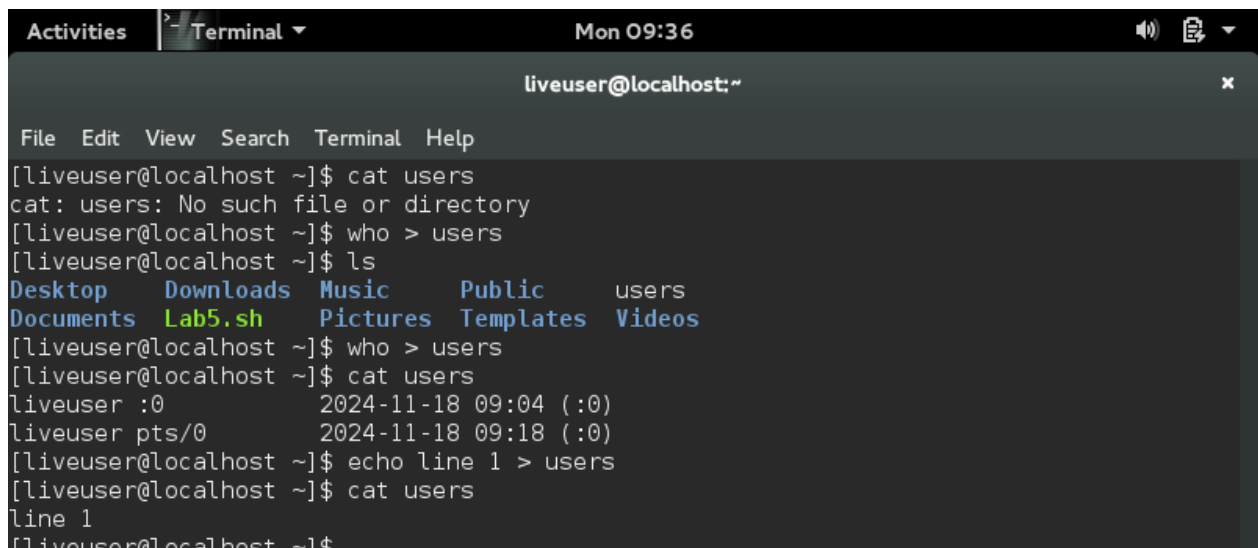
Output Redirection

The output from a command normally intended for standard output can be easily diverted to a file instead. This capability is known as output redirection:

If the notation **> file** is appended to any command that normally writes its output to standard output, the output of that command will be written to file instead of your terminal –

Check following **who** command which would redirect complete output of the command in users file.

```
$ who > users
```



```
liveuser@localhost:~  
File Edit View Search Terminal Help  
[liveuser@localhost ~]$ cat users  
cat: users: No such file or directory  
[liveuser@localhost ~]$ who > users  
[liveuser@localhost ~]$ ls  
Desktop Downloads Music Public users  
Documents Lab5.sh Pictures Templates Videos  
[liveuser@localhost ~]$ who > users  
[liveuser@localhost ~]$ cat users  
liveuser :0          2024-11-18 09:04 (:0)  
liveuser pts/0      2024-11-18 09:18 (:0)  
[liveuser@localhost ~]$ echo line 1 > users  
[liveuser@localhost ~]$ cat users  
line 1  
[liveuser@localhost ~]$
```

Notice that no output appears at the terminal. This is because the output has been redirected from the default standard output device (the terminal) into the specified file. If you would check *users* file, then it would have complete content –

```
[liveuser@localhost ~]$ cat user.sh
```

```
liveuser :0          2021-04-19 01:46 (:0)
```

```
liveuser pts/0      2021-04-19 01:56 (:0)
```

```
[liveuser@localhost ~]$ cat meta.sh
```

```
#!/bin/bash
```

```
echo hello \; world
```

```
echo "I have \$2000"
```

```
Date=`date`
```

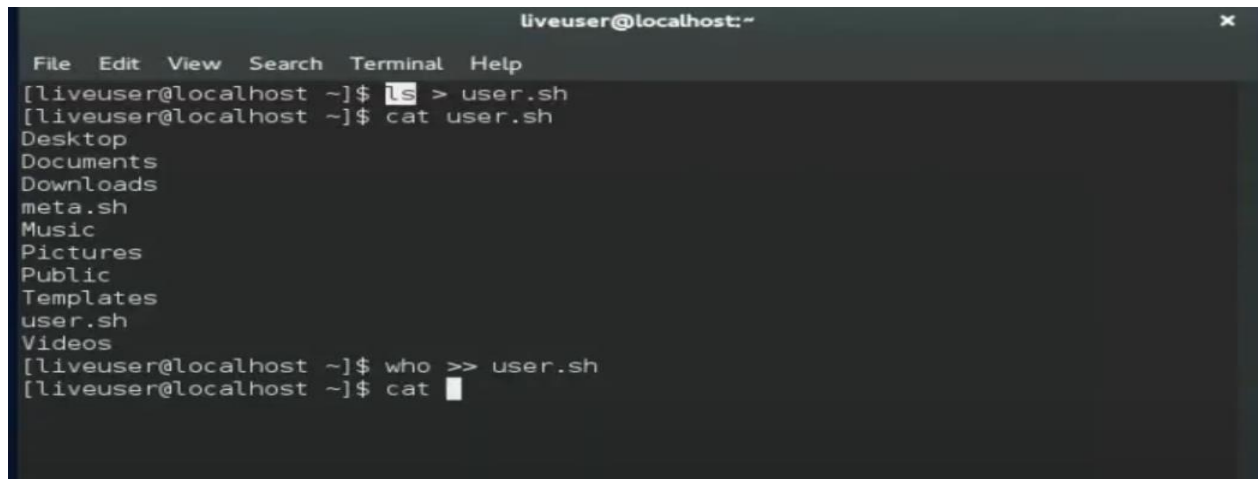
```
echo $Date
```

```
echo \<{\% \ hello i am special at \$ \# \ this moment \?
```

```
echo '<{% hello i am special at $ # \ this moment ?'
```

If a command has its output redirected to a file and the file already contains some data, that data will be lost. Consider this example –

```
$ echo line 1 > users
$ cat users
line 1
$
```

A screenshot of a terminal window titled 'liveuser@localhost:~'. The window has a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The terminal shows the following commands and output:

```
[liveuser@localhost ~]$ ls > user.sh
[liveuser@localhost ~]$ cat user.sh
Desktop
Documents
Downloads
meta.sh
Music
Pictures
Public
Templates
user.sh
Videos
[liveuser@localhost ~]$ who >> user.sh
[liveuser@localhost ~]$ cat
```

You can use >> operator to append the output in an existing file as follows –

```
$ echo line 2 >> users
$ cat users
line 1
line 2
$
```

```
[liveuser@localhost ~]$ date >> user.sh
[liveuser@localhost ~]$ pwd >> user.sh
[liveuser@localhost ~]$ cat user.sh
Desktop
Documents
Downloads
meta.sh
Music
Pictures
Public
Templates
user.sh
```

Input Redirection

Just as the output of a command can be redirected to a file, so can the input of a command be redirected from a file. As the greater-than character `>` is used for output redirection, the less-than character `<` is used to redirect the input of a command.

The commands that normally take their input from standard input can have their input redirected from a file in this manner. For example, to count the number of lines in the file *users* generated above, you can execute the command as follows –

```
$ wc -l users
2 users
$
```

Here it produces output 2 lines. You can count the number of lines in the file by redirecting the standard input of the `wc` command from the file *users* –

```
liveuser@localhost:~
File Edit View Search Terminal Help
[liveuser@localhost ~]$ wc user.sh
 14  27 215 user.sh
[liveuser@localhost ~]$ wc -l user.sh
14 user.sh
[liveuser@localhost ~]$ wc -l < user.sh
14
[liveuser@localhost ~]$
```

```
$ wc -l < users
2
$
```

Note that there is a difference in the output produced by the two forms of the `wc` command. In the first case, the name of the file `users` is listed with the line count; in the second case, it is not.

In the first case, we know that it is reading its input from the file `users`. In the second case, it only knows that it is reading its input from standard input so it does not display file name.

Here Document

A *here document* is used to redirect input into an interactive shell script or program.

We can run an interactive program within a shell script without user action by supplying the required input for the interactive program, or interactive shell script.

The general form for a here document is –

```
command << delimiter
document
delimiter
```

Here the shell interprets the `<<` operator as an instruction to read input until it finds a line containing the specified delimiter. All the input lines up to the line containing the delimiter are then fed into the standard input of the command.

The delimiter tells the shell that the here document has completed. Without it, the shell continues to read input forever. The delimiter must be a single word that does not contain spaces or tabs.

Following is the input to the command `wc -l` to count total number of line –

You can use *here document* to print multiple lines using your script as follows –

```
#!/bin/sh

cat << EOF

This is a simple look up program
for good (and bad) restaurants
in Cape Town.

EOF
```

```
[liveuser@localhost ~]$ ./Lab5.sh
This is a sample lookup program
for good (and bad) restaurants
in Cape Town.
[liveuser@localhost ~]$
```

The following script runs a session with the vi text editor and save the input in the file test.txt.

```
#!/bin/sh

filename=test.txt

vi $filename <<EndOfCommands

i

This file was created automatically from

a shell script

ZZ

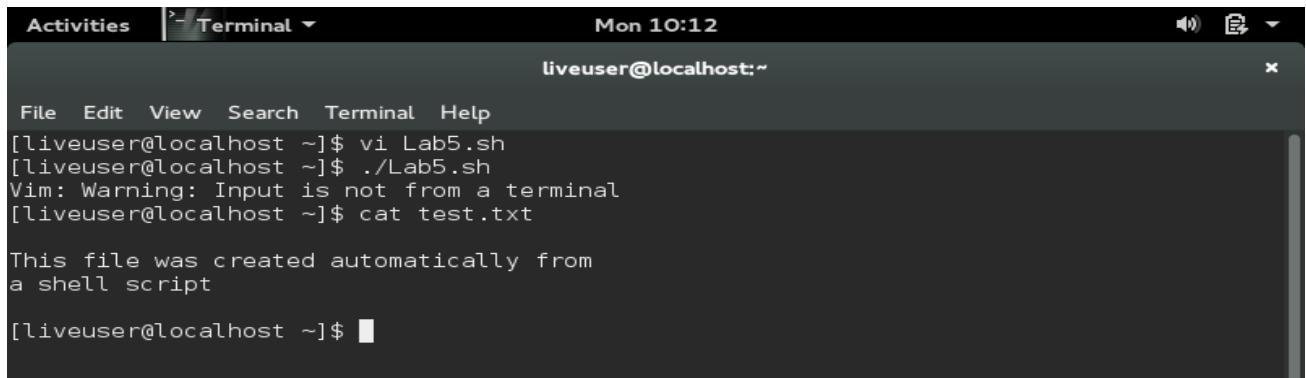
EndOfCommands
```

If you run this script with vim acting as vi, then you will likely see output like the following –

```
$ sh test.sh
Vim: Warning: Input is not from a terminal
$
```

After running the script, you should see the following added to the file test.txt –

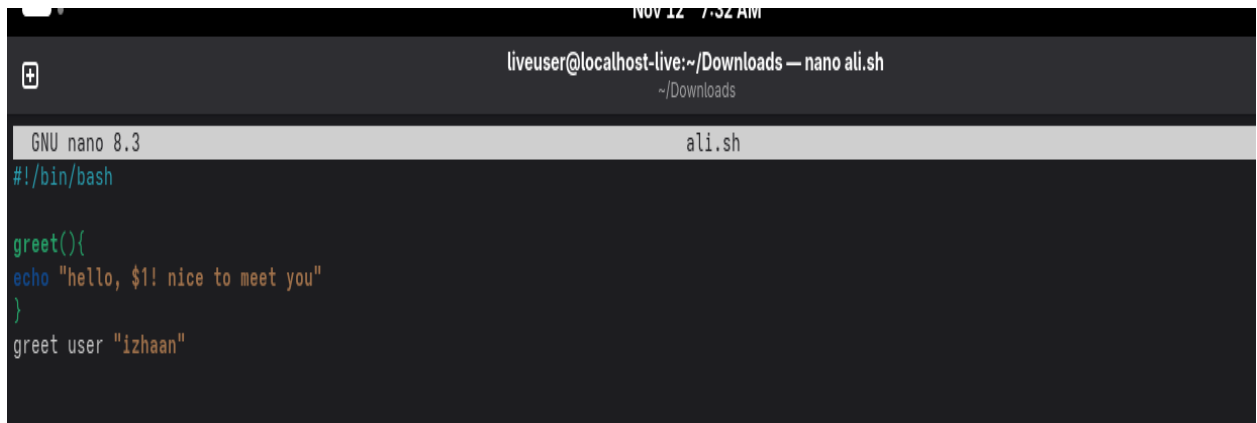
```
$ cat test.txt
This file was created automatically from
a shell script
$
```

A terminal window titled 'Terminal' with a timestamp 'Mon 10:12'. The prompt is 'liveuser@localhost:~'. The user enters 'vi Lab5.sh', then './Lab5.sh'. A warning message appears: 'Vim: Warning: Input is not from a terminal'. Then 'cat test.txt' is entered, displaying the output: 'This file was created automatically from a shell script'.

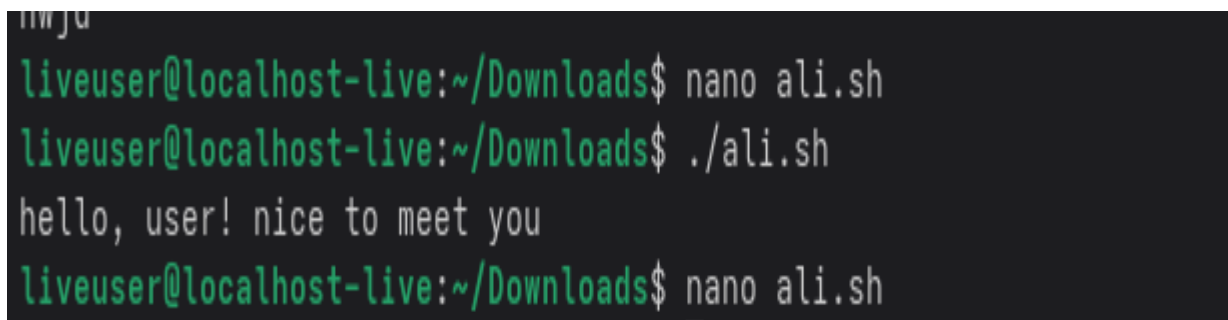
```
liveuser@localhost:~  
File Edit View Search Terminal Help  
[liveuser@localhost ~]$ vi Lab5.sh  
[liveuser@localhost ~]$ ./Lab5.sh  
Vim: Warning: Input is not from a terminal  
[liveuser@localhost ~]$ cat test.txt  
  
This file was created automatically from  
a shell script  
[liveuser@localhost ~]$
```

Task 1:

Use shell substitution to replace a variable with its value in a command (e.g., echo \$HOME).

A terminal window titled 'liveuser@localhost-live:~/Downloads — nano ali.sh'. It shows the GNU nano 8.3 editor with the file 'ali.sh'. The content of the file is a shell script: '#!/bin/bash', 'greet(){', 'echo "hello, \$! nice to meet you"', '}', and 'greet user "izhaan"'.

```
GNU nano 8.3 ali.sh  
#!/bin/bash  
  
greet(){  
echo "hello, $! nice to meet you"  
}  
greet user "izhaan"
```

A terminal window showing the execution of the script. The user runs 'nano ali.sh' and then './ali.sh'. The output is 'hello, user! nice to meet you'. The user then runs 'nano ali.sh' again.

```
liveuser@localhost-live:~/Downloads$ nano ali.sh  
liveuser@localhost-live:~/Downloads$ ./ali.sh  
hello, user! nice to meet you  
liveuser@localhost-live:~/Downloads$ nano ali.sh
```

Task 2:

Apply the use of single and double quotes in shell commands and explain their differences.

```
GNU nano 8.3
#!/bin/bash
add() {
sum=$(( $1 + $2 ))
return $sum
}
add 5 7
result=$?
echo "sum is: $result"
```

```
liveuser@localhost-live:~/Downloads$ nano kk.sh
liveuser@localhost-live:~/Downloads$ ./kk.sh
sum is: 12
liveuser@localhost-live:~/Downloads$
```

Task 3:

Construct the output of a command to a file using > and append to a file using >>.

```
GNU nano 8.3
#!/bin/bash
DATE=$(date)
echo "todays date is: $DATE"

USER_COUNT=$(who|wc -l)
echo "Numbers of users logged in:$USER_COUNT"
```

```
todays date is: Wed Nov 12 07:59:45 AM UTC 2025
Numbers of users logged in:2
liveuser@localhost-live:~/Downloads$
```

Task 4:

Use input redirection (<) to provide a file as input to a command (e.g., sort < file.txt).

```
GNU nano 8.3
```

```
#!/bin/bash
```

```
echo 'the value of $USER will not be displayed here.'
```

```
echo "the value of $USER is: $USER"
```

```
liveuser@localhost-live:~/Downloads$ chmod +x kk.sh
```

```
liveuser@localhost-live:~/Downloads$ ./kk.sh
```

```
/kk.sh: line 3: echothe value of $USER will not be displayed here.: command not found  
he value of$USER is: liveuser
```

Lab Session 6

Objective: To understand UNIX/Linux file management by creating, reading, editing and writing in file.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Introduction:

All data in UNIX is organized into files. All files are organized into directories. These directories are organized into a tree-like structure called the file system.

When you work with UNIX, one way or another you spend most of your time working with files. This tutorial would teach you how to create and remove files, copy and rename them, create links to them etc.

In UNIX there are three basic types of files –

Ordinary Files – An ordinary file is a file on the system that contains data, text, or program instructions. In this tutorial, you look at working with ordinary files.

Directories – Directories store both special and ordinary files. For users familiar with Windows or Mac OS, UNIX directories are equivalent to folders.

Special Files – Some special files provide access to hardware such as hard drives, CD-ROM drives, modems, and Ethernet adapters. Other special files are similar to Universityases or shortcuts and enable you to access a single file using different names.

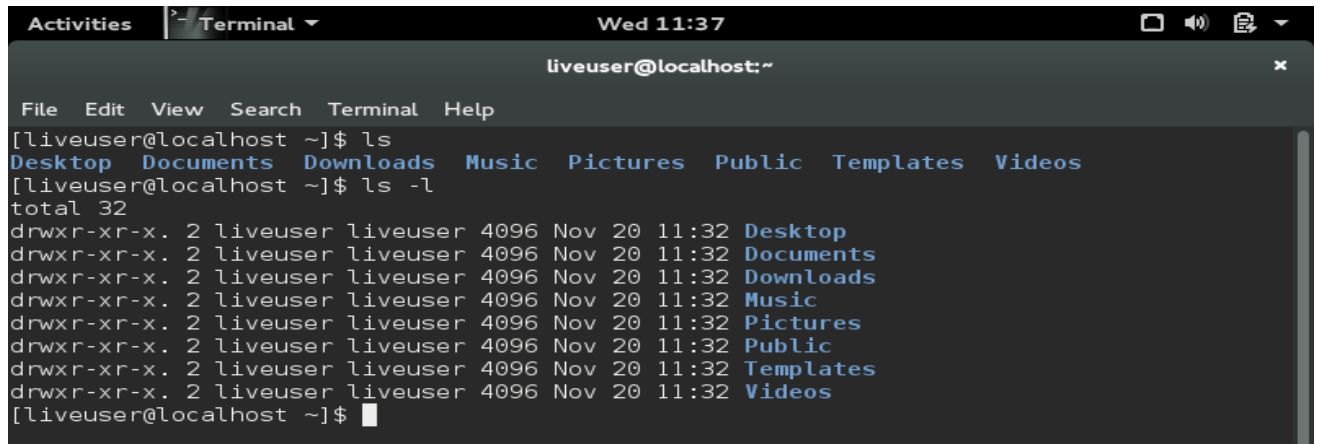
Listing Files

To list the files and directories stored in the current directory. Use the following command –

```
$ls
```

Here is the sample output of the above command –

The command `ls` supports the `-l` option which would help you to get more information about the listed files –

A screenshot of a Linux terminal window. The window title is "Terminal" and it shows the user "liveuser@localhost". The prompt is "[liveuser@localhost ~]\$". The user has entered the command "ls", and the output shows a list of directories: Desktop, Documents, Downloads, Music, Pictures, Public, Templates, and Videos. The user then enters the command "ls -l", and the output shows a detailed listing of these directories. Each line starts with "drwxr-xr-x.", followed by "2", "liveuser", "liveuser", "4096", "Nov 20 11:32", and the directory name. The prompt is "[liveuser@localhost ~]\$".

```
[liveuser@localhost ~]$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
[liveuser@localhost ~]$ ls -l
total 32
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Desktop
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Documents
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Downloads
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Music
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Pictures
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Public
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Templates
drwxr-xr-x. 2 liveuser liveuser 4096 Nov 20 11:32 Videos
[liveuser@localhost ~]$
```

Here is the information about all the listed columns –

- First Column: represents file type and permission given on the file. Below is the description of all type of files.
- Second Column: represents the number of memory blocks taken by the file or directory.
- Third Column: represents owner of the file. This is the Unix user who created this file.
- Fourth Column: represents group of the owner. Every Unix user would have an associated group.
- Fifth Column: represents file size in bytes.
- Sixth Column: represents date and time when this file was created or modified last time.
- Seventh Column: represents file or directory name.

In the `ls -l` listing example, every file line began with a `d`, `-`, or `l`. These characters indicate the type of file that's listed.

Prefix	Description
-	Regular file, such as an ASCII text file, binary executable, or hard link.
b	Block special file. Block input/output device file such as a physical hard drive.
c	Character special file. Raw input/output device file such as a physical hard drive
d	Directory file that contains a listing of other files and directories.
l	Symbolic link file. Links on any regular file.
p	Named pipe. A mechanism for interprocess communications
s	Socket used for interprocess communication.

Meta Characters

Meta characters have special meaning in Unix. For example, * and ? are metacharacters. We use * to match 0 or more characters, a question mark ? matches with single character.

For Example –

```
$ls ch*.doc
```

Displays all the files whose name start with ch and ends with .doc
–

```
ch01-1.doc ch010.doc ch02.doc ch03-2.doc
ch04-1.doc ch040.doc ch05.doc ch06-2.doc
ch01-2.doc ch02-1.doc c
```

Here * works as meta character which matches with any character. If you want to display all the files ending with just .doc then you can use following command –

```
$ls *.doc
```

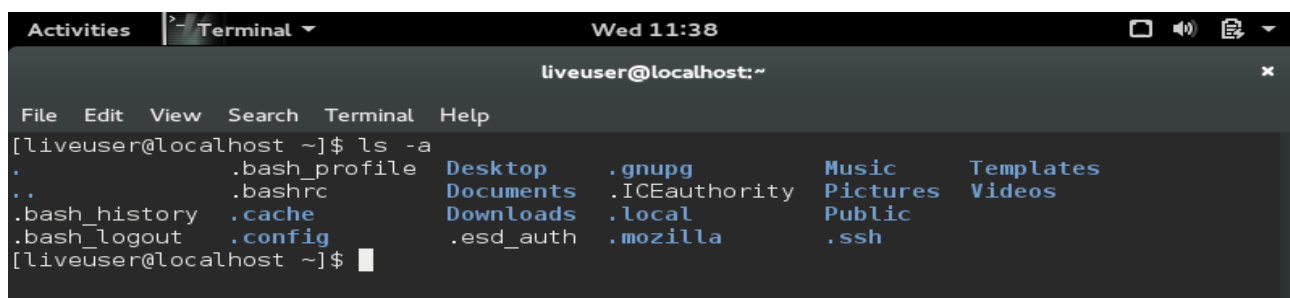
Hidden Files

An invisible file is one whose first character is the dot or period character (.). UNIX programs (including the shell) use most of these files to store configuration information.

Some common examples of hidden files include the files –

- `.profile` – the Bourne shell (`sh`) initialization script
- `.kshrc` – the Korn shell (`ksh`) initialization script
- `.cshrc` – the C shell (`csh`) initialization script
- `.rhosts` – the remote shell configuration file

To list invisible files, specify the `-a` option to `ls` –



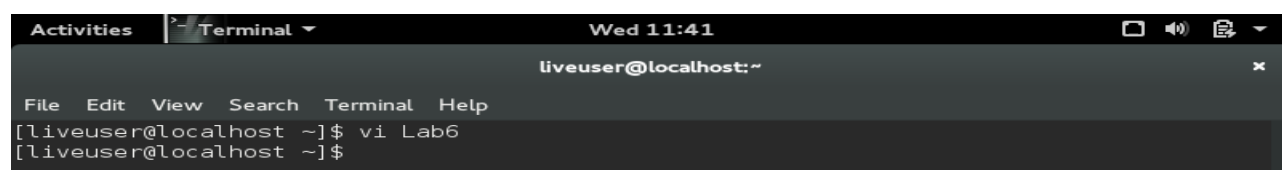
```
liveuser@localhost:~  
File Edit View Search Terminal Help  
[liveuser@localhost ~]$ ls -a  
.      .bash_profile  Desktop      .gnupg      Music      Templates  
..     .bashrc       Documents    .ICEauthority Pictures    Videos  
      .bash_history .cache       Downloads    .local      Public  
      .bash_logout .config      .esd_auth    .mozilla    .ssh  
[liveuser@localhost ~]$
```

- Single dot `.` – This represents current directory.
- Double dot `..` – This represents parent directory.

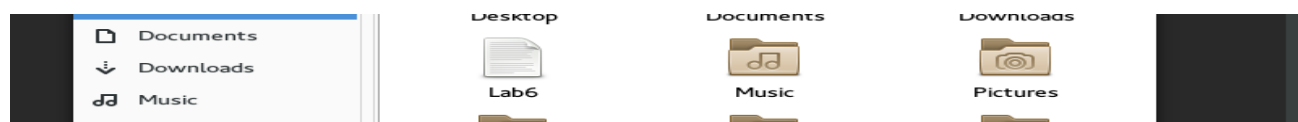
Creating Files

You can use `vi` editor to create ordinary files on any Unix system. You simply need to give following command –

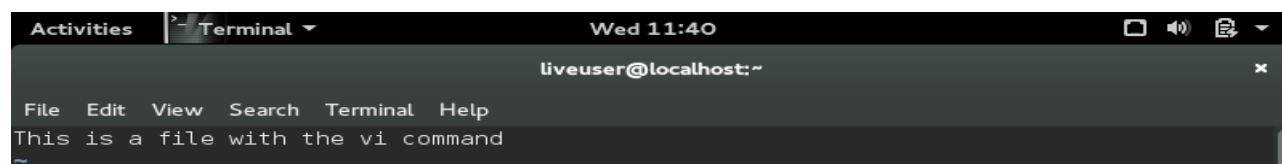
```
$ vi filename
```



```
liveuser@localhost:~  
File Edit View Search Terminal Help  
[liveuser@localhost ~]$ vi Lab6  
[liveuser@localhost ~]$
```



Above command would open a file with the given filename. You would need to press key `i` to come into edit mode. Once you are in edit mode you can start writing your content in the file as below –



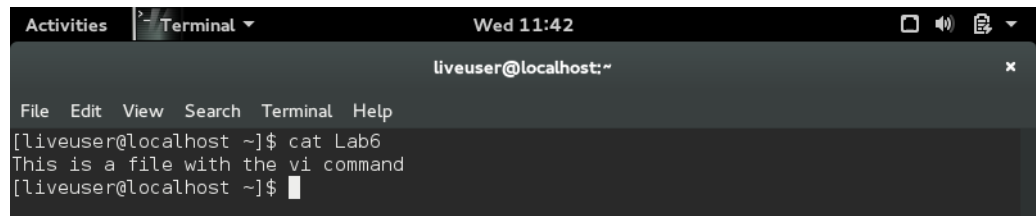
```
liveuser@localhost:~  
File Edit View Search Terminal Help  
This is a file with the vi command  
~
```

Once you are done, do the following steps –

- Press key **esc** to come out of edit mode.
- Press two keys **Shift + ZZ** together to come out of the file completely.

Now you would have a file created with **filename** in the current directory.

```
$ vi filename  
$
```



Editing Files

You can edit an existing file using **vi** editor. We would cover this in detail in a separate tutorial. But in short, you can open existing file as follows –

```
$ vi filename
```

Once file is opened, you can come in edit mode by pressing key **i** and then you can edit file as you like. If you want to move here and there inside a file then first you need to come out of edit mode by pressing key **esc** and then you can use following keys to move inside a file –

- **l** key to move to the right side.
- **h** key to move to the left side.
- **k** key to move up side in the file.
- **j** key to move down side in the file.

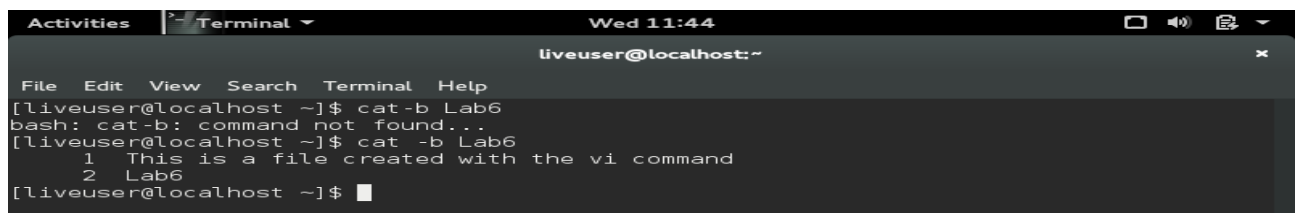
So using above keys you can position your cursor where ever you want to edit. Once you are positioned then you can use **i** key to come in edit mode. Edit the file, once you are done press **esc** and finally two keys **Shift + ZZ** together to come out of the file completely.

Display Content of a File

You can use **cat** command to see the content of a file. Following is the simple example to see the content of above created file –

```
$ cat filename
This is unix file....I created it for the first time.....
I'm going to save this content in this file.
$
```

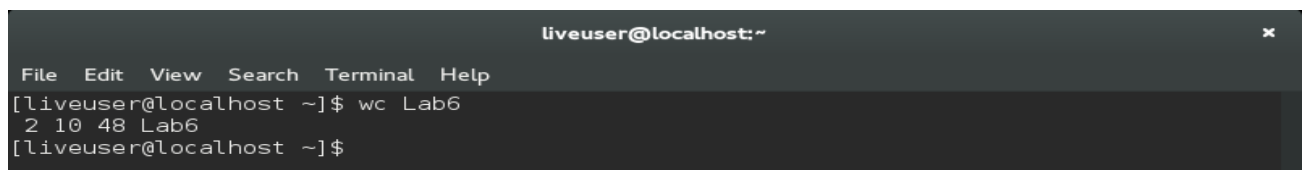
You can display line numbers by using **-b** option along with **cat** command as follows
cat -b Lab6

A terminal window titled 'Terminal' with a menu bar (File, Edit, View, Search, Terminal, Help) and a prompt 'liveuser@localhost:~'. The user enters 'cat -b Lab6', which results in an error 'bash: cat -b: command not found...'. Then, the user enters 'cat -b Lab6', which displays the file content with line numbers: '1 This is a file created with the vi command' and '2 Lab6'.

```
liveuser@localhost:~
File Edit View Search Terminal Help
[liveuser@localhost ~]$ cat -b Lab6
bash: cat -b: command not found...
[liveuser@localhost ~]$ cat -b Lab6
1 This is a file created with the vi command
2 Lab6
[liveuser@localhost ~]$
```

Counting Words in a File

You can use the **wc** command to get a count of the total number of lines, words, and characters contained in a file. Following is the simple example to see the information about above created file.

A terminal window titled 'Terminal' with a menu bar (File, Edit, View, Search, Terminal, Help) and a prompt 'liveuser@localhost:~'. The user enters 'wc Lab6', which displays the file statistics: '2 10 48 Lab6'.

```
liveuser@localhost:~
File Edit View Search Terminal Help
[liveuser@localhost ~]$ wc Lab6
2 10 48 Lab6
[liveuser@localhost ~]$
```

Here is the detail of all the four columns –

- First Column: represents total number of lines in the file.
- Second Column: represents total number of words in the file.
- Third Column: represents total number of bytes in the file. This is actual size of the file.
- Fourth Column: represents file name.

You can give multiple files at a time to get the information about those file. Here is simple syntax –

```
$ wc filename1 filename2 filename3
```

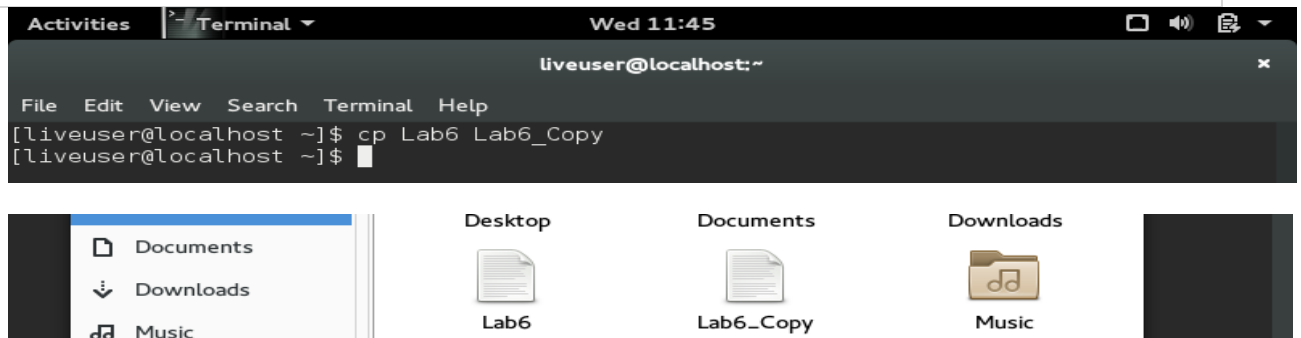
Copying Files:

To make a copy of a file use the **cp** command. The basic syntax of the command is

```
$ cp source_file destination_file
```

Following is the example to create a copy of existing file **filename**.

```
$ cp filename copyfile
$
```



Now you would find one more file **copy file** in your current directory. This file would be exactly same as original file **filename**.

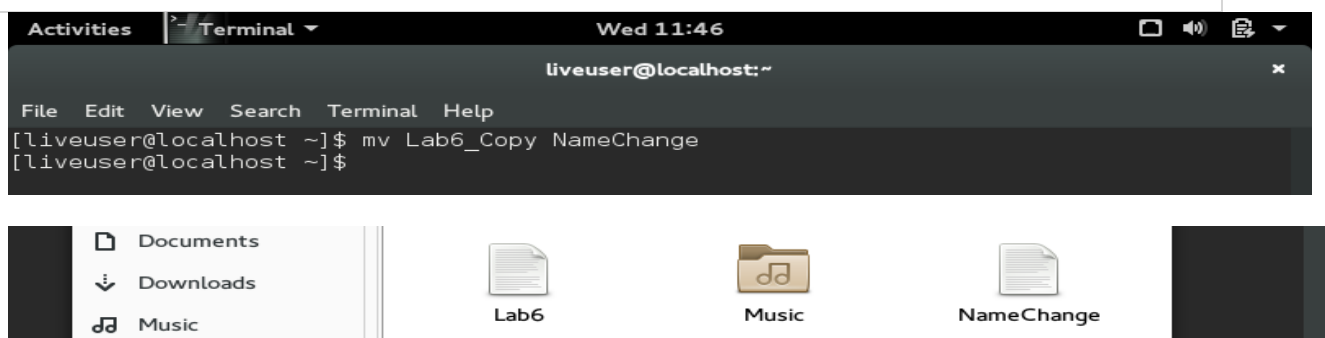
Renaming Files

To change the name of a file use the **mv** command. Its basic syntax is –

```
$ mv old_file new_file
```

Following is the example which would rename existing file **filename** to **newfile**:

```
$ mv filename newfile
$
```



The **mv** command would move existing file completely into new file. So in this case you would find only **newfile** in your current directory.

Deleting Files

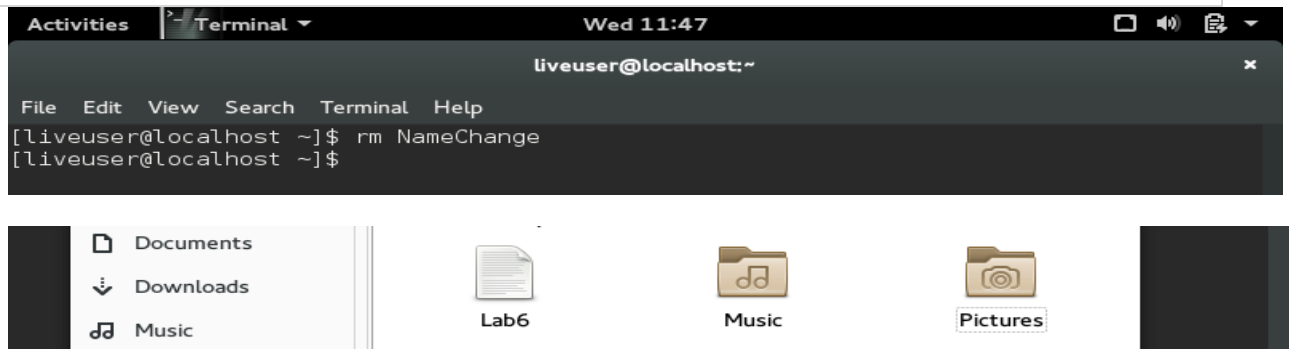
To delete an existing file, use the **rm** command. Its basic syntax is –

```
$ rm filename
```

Caution: It may be dangerous to delete a file because it may contain useful information. So be careful while using this command. It is recommended to use **-i** option along with **rm** command.

Following is the example which would completely remove existing file **filename**:

```
$ rm filename  
$
```

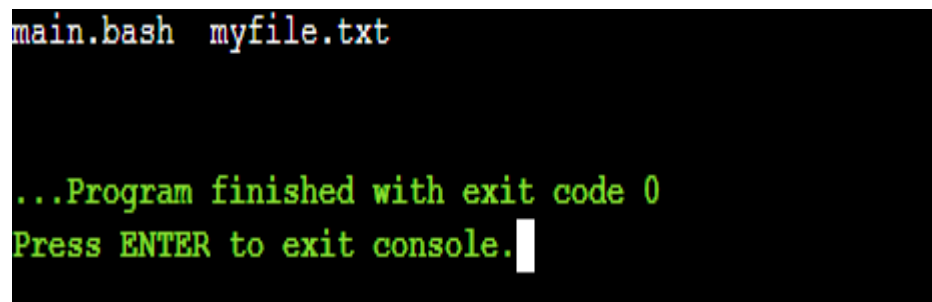
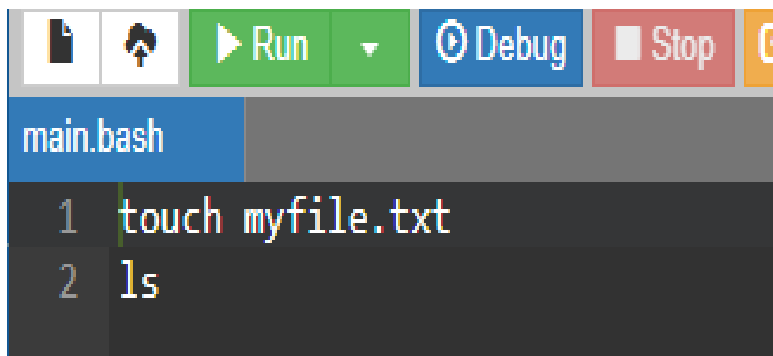


You can remove multiple files at a time as follows –

```
$ rm filename1 filename2 filename3  
$
```

Task 1:

Construct a new text file using the touch command and verify its creation with ls.



Task 2:

Rename a file using the mv command and verify the change.

```
main.bash  myfile.txt  ⋮
1 mv myfile.txt newfile.txt
2 ls
```

```
main.bash  myfile.txt

...Program finished with exit code 0
Press ENTER to exit console.
```

Task 3:

Delete a file using the rm command and confirm its removal with ls.

```
main.bash  myfile.txt  ⋮
1 rm newfile.txt
2 ls
```

```
main.bash  myfile.txt

...Program finished with exit code NaN
Press ENTER to exit console.
```

Lab Session 7 (Open-Ended Lab)

Assignment of Open-Ended Lab / CCA / PBL

Blooms Taxonomy	GAs	Knowledge Profile	EAs	WPs
C3 A3	GA-2 GA-6	WK5	EA5	WP5

Title: Design and Development of a Management System Using Linux Fedora

Motivation:

Operating systems are vital in managing resources and enabling user interactions in computing systems. Linux Fedora, a widely used open-source OS, provides an excellent platform for students to explore real-world system design challenges. This lab encourages students to deepen their understanding of core OS concepts like process management, file systems, and resource allocation while developing practical problem-solving skills.

Concept (Problem Statement):

Students will design and implement a management system using Linux Fedora that addresses a real-world need. The system should utilize core Fedora features and demonstrate the application of operating system principles. Examples of management systems include:

1. System Resource Manager: Monitor and manage system resources (CPU, memory, or disk usage) effectively.
2. Process Management System: Create a tool to handle process operations like monitoring, filtering, and terminating processes.
3. File Management System: Design a system for managing files and directories, including search, backup, and permission settings.
4. Task Automation Manager: Develop a tool for scheduling and automating tasks based on user-defined priorities or time constraints.
5. User Management System: Implement a system for creating, managing, and securing user accounts.

Deliverables

Background/Theory:

Procedure / Methodology:

Data Collection (If required):

Flowchart / Block diagram:

Analysis:

Results:

Discussion on Results:

Concluding Remarks:

Reference:

Lab Session 8

Objective: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithms to find turnaround time and waiting time.

- FCFS
- SJF
- Round Robin (pre-emptive)
- Priority

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario.

- All the processes in the system are divided into two categories – system processes and user processes.
- System processes are to be given higher priority than user processes.

Use FCFS scheduling for the processes in each queue.

Required Equipment / tools:

1. Dev C++

Introduction: In computer science, CPU scheduling is a fundamental concept in operating systems that ensures efficient execution of processes by determining the order in which they access the CPU. Scheduling algorithms play a pivotal role in optimizing system performance, balancing workloads, and reducing delays. This project focuses on implementing and simulating both non-preemptive and preemptive scheduling algorithms in C to compute key performance metrics such as Turnaround Time and Waiting Time.

The algorithms implemented in this program include:

First Come First Serve (FCFS): A straightforward approach where processes are executed in the order of their arrival.

Shortest Job First (SJF): Prioritizes processes with the shortest burst time, minimizing the overall turnaround time.

Round Robin (Pre-emptive): Allocates CPU time in a time-sharing manner, ensuring fair distribution among processes.

Priority Scheduling: Processes are scheduled based on assigned priorities, with higher-priority processes being executed first.

What Does "Non-Preemptive" Mean?

Definition (Simple Words):

Non-preemptive scheduling means once a process starts execution, it cannot be stopped until it finishes.

No interruption. No switching. No force-stopping.

It runs completely → then the CPU moves to the next process.

Real-Life Example (Very Easy):

Imagine you are waiting at a barber shop.

Non-Preemptive = Once the barber starts cutting someone's hair, he will finish fully.

Even if:

Another customer comes (even with higher priority)

Or someone is in a hurry

Or a VIP enters

Barber WILL NOT stop the current customer.

In CPU Terms:

A process enters the CPU

It gets full CPU access

It runs until completion

Only then the next process gets CPU

Technical Explanation

Non-Preemptive = No context switching until process finishes

CPU does not take back control once it is given

Process voluntarily releases the CPU when:

- It completes
- It performs I/O operation

CPU DOES NOT interrupt on its own.

Why Non-Preemptive Scheduling Exists?

- Simple to implement
- Less overhead (no context switching)
- Works well when tasks have different arrival times
- Useful for systems where stability is more important than responsiveness

Disadvantages of Non-Preemptive

- Long processes can block short processes
- Can increase average waiting time
- Not good for interactive systems (like OS of computers and mobile)

Examples of Non-Preemptive Scheduling Algorithms

FCFS (First Come First Served)

First process gets all CPU, no interruption

Simple queue method

SJF (Shortest Job First) – Non-Preemptive version

Pick the shortest job

Once it starts → cannot be stopped until completed

Priority Scheduling (Non-Preemptive)

Highest priority process starts

Cannot be preempted by a new higher-priority process

Difference Between Preemptive vs Non-Preemptive

Feature	Preemptive	Non-Preemptive
Interruption	Allowed	Not allowed
CPU Control	OS can forcefully take CPU CPU stays with process until finish	

Feature	Preemptive	Non-Preemptive
Response Time	Fast	Slow
Context Switching	High overhead	Low overhead
Used In	Interactive systems, OS	Batch systems

Non-Preemptive Scheduling Timeline Example

Suppose we have processes:

Process Burst Time

P1 6

P2 4

P3 2

In FCFS (Non-Preemptive):

|---P1---|---P2---|---P3---|
0 6 10 12

Even if P3 (2 sec) is shorter, it waits because non-preemptive FCFS does NOT allow switching.

FCFS (First Come First Served)

- Non-preemptive
- The process that arrives first gets CPU first
- Works like a normal queue
- Waiting Time = time spent in queue
- Turnaround Time = waiting time + burst time

SJF (Shortest Job First)

- Non-preemptive
- The process with the smallest burst time is executed first
- Minimizes average waiting time
- Can cause starvation for long processes

Round Robin (Preemptive)

- Each process gets a fixed time quantum
- When quantum expires, process goes to the back of the queue
- Ensures fairness
- Used in time-sharing systems

Round Robin Scheduling (Preemptive) – C Program

```
#include <stdio.h>

int main() {
    int n, quantum;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n]; // rt = remaining time

    printf("Enter burst times:\n");
    for(int i = 0; i < n; i++) {
        printf("P%d: ", i+1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i]; // initialize remaining time
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    int t = 0; // current time
    int done;

    do {
        done = 1; // assume all processes finished

        for(int i = 0; i < n; i++) {
```

```

done = 0;

if(rt[i] > quantum) {
    t += quantum;
    rt[i] -= quantum;
} else {
    t += rt[i];
    printf("Process P%d finished at time %d\n", i+1, t);
    rt[i] = 0;
}

}

}

} while(!done); // repeat until all processes done

```

Outputs:

The screenshot shows a code editor with a C program on the left and its output on the right. The code implements a round-robin scheduling algorithm. The output shows the user inputting 4 processes with burst times 5, 2, 3, and 6, and a quantum of 2. The program then prints the finish times for each process: P2 at 4, P3 at 11, P1 at 14, and P4 at 16. The execution is successful.

```

if(rt[i] > 0) { // process still running
    done = 0;

    if(rt[i] > quantum) {
        t += quantum;
        rt[i] -= quantum;
    } else {
        t += rt[i];
        printf("Process P%d finished at time %d\n", i+1,
            t);
        rt[i] = 0;
    }
}
}
}

Enter number of processes: 4
Enter burst times:
P1: 5
P2: 2
P3: 3
P4: 6
Enter time quantum: 2
Process P2 finished at time 4
Process P3 finished at time 11
Process P1 finished at time 14
Process P4 finished at time 16

=== Code Execution Successful ===

```

Round Robin Scheduling (Preemptive) – C Program

```
#include <stdio.h>

int main() {
    int n, quantum;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], rt[n]; // rt = remaining time

    printf("Enter burst times:\n");
    for(int i = 0; i < n; i++) {
        printf("P%d: ", i+1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i]; // initialize remaining time
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    int t = 0; // current time
    int done;

    do {
        done = 1; // assume all processes finished

        for(int i = 0; i < n; i++) {

            if(rt[i] > 0) { // process still running
                done = 0;
                if(rt[i] > quantum) { t += quantum;
```

```
        t += rt[i];
        printf("Process P%d finished at time %d\n", i+1, t);
        rt[i] = 0;
    }

}
```

```
} while(!done); // repeat until all processes done
```

Output:

```
Enter number of processes: 5
```

```
Enter burst times:
```

```
P1: 2
```

```
P2: 3
```

```
P3: 1
```

```
P4: 4
```

```
P5: 5
```

```
Enter time quantum: 3
```

```
P1 finished at time 2
```

```
P2 finished at time 5
```

```
P3 finished at time 6
```

```
P4 finished at time 13
```

```
P5 finished at time 15
```

```
=== Code Execution Successful ===
```

Acti
Get

Priority Scheduling (Non-Preemptive) – C Program

```
#include <stdio.h>

int main() {
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], pr[n], p[n], wt[n], tat[n];

    for(int i = 0; i < n; i++) {
        printf("Enter burst time and priority of P%d: ", i+1);
        scanf("%d %d", &bt[i], &pr[i]);
        p[i] = i + 1;
    }

    // sorting by priority
    for(int i = 0; i < n; i++) {
        for(int j = i+1; j < n; j++) {
            if(pr[j] < pr[i]) {
                int temp;

                temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    ,
```

```

for(int i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}

for(int i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i], pr[i], bt[i], wt[i], tat[i]);
}

```

Output:

```

Enter number of processes: 6
Enter burst time and priority of P1: 1
2
Enter burst time and priority of P2: 2
3
Enter burst time and priority of P3: 3
4
Enter burst time and priority of P4: 4
5
Enter burst time and priority of P5: 5
6
Enter burst time and priority of P6: 6
5

```

Process	Priority	BT	WT	TAT
P1	2	1	0	1
P2	3	2	1	3
P3	4	3	3	6
P4	5	4	6	10
P6	5	6	10	16
P5	6	5	16	21

```

=== Code Execution Successful ===

```

Multilevel Queue Scheduling – C Program

```
#include <stdio.h>

int main() {
    int s, u;

    printf("Enter number of System processes: ");
    scanf("%d", &s);

    printf("Enter number of User processes: ");
    scanf("%d", &u);

    int sysBT[s], userBT[u];

    printf("\nEnter burst times for System processes:\n");
    for(int i = 0; i < s; i++) {
        printf("S%d: ", i+1);
        scanf("%d", &sysBT[i]);
    }

    printf("\nEnter burst times for User processes:\n");
    for(int i = 0; i < u; i++) {
        printf("U%d: ", i+1);
        scanf("%d", &userBT[i]);
    }
}
```

```
int time = 0;

printf("\n--- Executing System Queue (High Priority) ---\n");
for(int i = 0; i < s; i++) {
    time += sysBT[i];
    printf("System Process S%d finished at time %d\n", i+1, time);
}

printf("\n--- Executing User Queue (Low Priority) ---\n");
for(int i = 0; i < u; i++) {
    time += userBT[i];
    printf("User Process U%d finished at time %d\n", i+1, time);
}
```

Lab Session 9

Objective: Understanding and Simulating Producer Consumer Problem and Dining Philosopher Problem.

1. Write a C program to simulate producer-consumer problem using semaphores.
2. Write a C program to simulate the concept of Dining-Philosophers problem

Required Equipment / tools:

1. Dev C++

Producer–Consumer Problem

1. Race Condition

- When two or more threads/processes access shared data **at the same time**.
- If one changes the data before the other finishes, results become **incorrect or unpredictable**.
- Synchronization (mutex/semaphores) prevents race conditions.

2. Critical Section

- A portion of code where shared data (buffer) is accessed.
- Only **one thread** should access it at a time.

3. Mutual Exclusion

- Ensures that only **one thread** enters the critical section at any moment.
- Achieved using **mutex** or **binary semaphore**.

4. Semaphore

- A synchronization tool used to manage resource availability.
- Types:
 - **Binary semaphore (mutex)** → only 0 or 1
 - **Counting semaphore (empty/full)** → counts available slots/items

5. Bounded Buffer

- Buffer with **fixed size** where producer adds items and consumer removes them.

```

#include <stdio.h>      // Standard input/output library (printf)
#include <pthread.h>    // For creating and managing threads
#include <semaphore.h>  // For using semaphores (sem_wait, sem_post)
#include <unistd.h>     // For sleep() function

#define BUFFER_SIZE 5   // Defining buffer size as 5 (bounded buffer)

// Shared buffer and index variables
int buffer[BUFFER_SIZE]; // Circular buffer shared between producer & consumer
int in = 0, out = 0;      // 'in' = next insertion index, 'out' = next removal index

// Semaphores declaration
sem_t empty;              // Counts empty slots in buffer
sem_t full;               // Counts filled slots
sem_t mutex;              // Ensures mutual exclusion (avoids race condition)

// Producer thread function
void *producer(void *arg) {
    int item = 1;          // First produced item starts from 1

    while (1) {            // Infinite loop to continuously produce items

        sem_wait(&empty);  // Wait (block) if buffer is full
        sem_wait(&mutex);  // Lock the critical section (mutual exclusion)

        buffer[in] = item; // Insert item into buffer at index 'in'
        printf("Producer produced: %d at %d\n", item, in);

        in = (in + 1) % BUFFER_SIZE; // Move 'in' index circularly
        item++;                      // Increase item value for next production

        sem_post(&mutex); // Unlock critical section
        sem_post(&full);  // Increase full count (consumer can consume now)

        sleep(1);          // Pause production for 1 second (simulating delay)
    }
}

// Consumer thread function
void *consumer(void *arg) {

    while (1) {            // Infinite loop to continuously consume items

        sem_wait(&full);   // Wait (block) if buffer is empty
        sem_wait(&mutex);  // Lock critical section for safe consumption

        int item = buffer[out]; // Fetch item from buffer
        printf("Consumer consumed: %d from %d\n", item, out);

        out = (out + 1) % BUFFER_SIZE; // Move 'out' index circularly

        sem_post(&mutex); // Unlock critical section
    }
}

```

```

    sem_post(&empty); // Increase empty slot count (producer can produce)

    sleep(2);        // Pause consumption for 2 seconds (slower consumer)
}
}

int main() {
    pthread_t p, c;    // Thread identifiers for producer & consumer

    // Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE); // 'empty' starts with all buffer slots empty
    sem_init(&full, 0, 0);           // 'full' starts with 0 (nothing to consume)
    sem_init(&mutex, 0, 1);          // 'mutex' starts unlocked (1)

    // Create threads for producer and consumer
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    // Wait for threads to finish (they never will because of infinite loops)
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}

```

Dining Philosophers Problem

What is the Dining Philosophers Problem?

- A classical **synchronization problem** in Operating Systems.
- Demonstrates the challenges of **resource sharing** among multiple processes.

Scenario Description

- 5 philosophers sit around a round table.
- Between each pair of philosophers there is **one fork**.
- Each philosopher repeats two activities:
 - **Thinking**
 - **Eating**
- To eat, a philosopher must pick up:
 - The **left fork**
 - The **right fork**

Main Issues Addressed

1. Deadlock

- All philosophers pick the left fork → no one gets the right fork → system stuck.

2. Starvation

- A philosopher may never get the chance to eat.

3. Concurrency

- Multiple philosophers can eat at the same time as long as forks do not clash.

Learning Objective

- Use semaphores or mutual exclusion to:
 - Avoid deadlock
 - Ensure fair access to forks
 - Manage synchronization properly

```
#include <stdio.h>      // Standard input/output functions
#include <pthread.h>     // For using threads
#include <semaphore.h>   // For semaphore functions
#include <unistd.h>      // For sleep() function

#define N 5             // Number of philosophers and forks

sem_t forks[N];         // One semaphore per fork (each fork is a shared resource)

// Thread function for each philosopher
void *philosopher(void *num) {
    int id = *(int *)num; // Extract philosopher ID (0–4)

    while (1) {
        printf("Philosopher %d is thinking...\n", id);
        sleep(1);         // Thinking time simulation

        // Pick up the left fork
        sem_wait(&forks[id]);
        printf("Philosopher %d picked up LEFT fork %d\n", id, id);

        // Pick up the right fork (circular access)
        sem_wait(&forks[(id + 1) % N]);
        printf("Philosopher %d picked up RIGHT fork %d\n", id, (id + 1) % N);

        // Eating
        printf("Philosopher %d is eating...\n", id);
        sleep(1);         // Eating time simulation
```

```

    // Put down left fork
    sem_post(&forks[id]);
    printf("Philosopher %d put down LEFT fork %d\n", id, id);

    // Put down right fork
    sem_post(&forks[(id + 1) % N]);
    printf("Philosopher %d put down RIGHT fork %d\n", id, (id + 1) % N);

    // After eating, philosopher returns to thinking
    printf("Philosopher %d finished eating and starts thinking again...\n", id);
    sleep(1);
}
}

int main() {
    pthread_t philosophers[N]; // Array of threads for philosophers
    int ids[N];               // To store philosopher IDs

    // Initialize each fork semaphore with value = 1 (available)
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1); // 1 → fork is free
        ids[i] = i;               // Assign philosopher ID
    }

    // Create threads for each philosopher
    for (int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads (infinite loop, never ends)
    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    return 0;
}

```

Task 1 — Multi-Producer, Multi-Consumer Messaging System

Scenario

A social media backend system receives messages from 3 different servers and delivers them to 2 client applications.

Student Task

Modify the Producer–Consumer program to:

- Create 3 producer threads (servers)
- Create 2 consumer threads (client applications)
- Each message should include:
 - Producer ID
 - Message number
- Display buffer status after each insert/remove
- Prevent race conditions using semaphores and mutex

Code:

```

1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <condition_variable>
5
6  using namespace std;
7
8  const int BUFFER_SIZE = 5;
9
10 int buffer[BUFFER_SIZE];
11 int in = 0;
12 int out = 0;
13 int count = 0;
14
15 mutex mtx;
16 condition_variable emptyCV, fullCV;
17
18 int messageNumber = 1;
19
20 // Display Buffer
21 void displayBuffer() {
22
23     cout << "Buffer: ";
24
25     for(int i = 0; i < BUFFER_SIZE; i++) {
26         cout << buffer[i] << " ";
27     }
28
29     cout << endl;
30 }
31
32 // Producer Function
33 void producer(int id) {
34
35     for(int i = 0; i < 5; i++) {

```

```

35     for(int i = 0; i < 5; i++) {
36
37         unique_lock<mutex> lock(mtx);
38
39         emptyCV.wait(lock, [] {
40             return count < BUFFER_SIZE;
41         });
42
43         int msg = messageNumber++;
44
45         buffer[in] = msg;
46
47         cout << "Producer " << id
48             << " produced Message " << msg
49             << " at position " << in << endl;
50
51         in = (in + 1) % BUFFER_SIZE;
52
53         count++;
54
55         displayBuffer();
56
57         lock.unlock();
58
59         fullCV.notify_all();
60     }
61 }
62
63 // Consumer Function
64 void consumer(int id) {
65
66     for(int i = 0; i < 7; i++) {
67
68         unique_lock<mutex> lock(mtx);
69

```

```

70     fullCV.wait(lock, [] {
71         return count > 0;
72     });
73
74     int msg = buffer[out];
75
76     buffer[out] = 0;
77
78     cout << "Consumer " << id
79         << " consumed Message " << msg
80         << " from position " << out << endl;
81
82     out = (out + 1) % BUFFER_SIZE;
83
84     count--;
85
86     displayBuffer();
87
88     displayBuffer();
89
90     lock.unlock();
91     emptyCV.notify_all();
92 }
93
94 int main() {
95
96     // Initialize buffer
97     for(int i = 0; i < BUFFER_SIZE; i++) {
98         buffer[i] = 0;
99     }
100
101     // Producer Threads
102     thread p1(producer, 1);

```

```

102     thread p1(producer, 1);
103     thread p2(producer, 2);
104     thread p3(producer, 3);
105
106     // Consumer Threads
107     thread c1(consumer, 1);
108     thread c2(consumer, 2);
109
110     // Join Threads
111     p1.join();
112     p2.join();
113     p3.join();
114
115     c1.join();
116     c2.join();
117
118     return 0;
119 }

```

Output:

```

Producer 1 produced Message 1 at position 0
Buffer: 1 0 0 0 0
Producer 1 produced Message 2 at position 1
Buffer: 1 2 0 0 0
Producer 1 produced Message 3 at position 2
Buffer: 1 2 3 0 0
Producer 1 produced Message 4 at position 3
Buffer: 1 2 3 4 0
Producer 1 produced Message 5 at position 4
Buffer: 1 2 3 4 5
Consumer 1 consumed Message 1 from position 0
Buffer: 0 2 3 4 5
Consumer 1 consumed Message 2 from position 1
Buffer: 0 0 3 4 5
Producer 3 produced Message 6 at position 0
Buffer: 6 0 3 4 5
Consumer 2 consumed Message 3 from position 2
Buffer: 6 0 0 4 5

```

Activate Window

```
Consumer 2 consumed Message 3 from position 2
Buffer: 6 0 0 4 5
Consumer 2 consumed Message 4 from position 3
Buffer: 6 0 0 0 5
Consumer 2 consumed Message 5 from position 4
Buffer: 6 0 0 0 0
Consumer 2 consumed Message 6 from position 0
Buffer: 0 0 0 0 0
Producer 2 produced Message 7 at position 1
Buffer: 0 7 0 0 0
Producer 2 produced Message 8 at position 2
Buffer: 0 7 8 0 0
Producer 2 produced Message 9 at position 3
Buffer: 0 7 8 9 0
Producer 2 produced Message 10 at position 4
Buffer: 0 7 8 9 10
Consumer 2 consumed Message 7 from position 1
Buffer: 0 0 8 9 10
Buffer: 0 0 8 9 10
Consumer 2 consumed Message 8 from position 2
Buffer: 0 0 0 9 10
Consumer 1 consumed Message 9 from position 3
Buffer: 0 0 0 0 10
Consumer 1 consumed Message 10 from position 4
Buffer: 0 0 0 0 0
Producer 3 produced Message 11 at position 0
Buffer: 11 0 0 0 0
Producer 3 produced Message 12 at position 1
Buffer: 11 12 0 0 0
Producer 3 produced Message 13 at position 2
Buffer: 11 12 13 0 0
Producer 3 produced Message 14 at position 3
Buffer: 11 12 13 14 0
Consumer 1 consumed Message 11 from position 0
Buffer: 0 12 13 14 0
Consumer 1 consumed Message 12 from position 1
```

```
Consumer 1 consumed Message 11 from position 0
Buffer: 0 12 13 14 0
Consumer 1 consumed Message 12 from position 1
Buffer: 0 0 13 14 0
Consumer 1 consumed Message 13 from position 2
Buffer: 0 0 0 14 0
Consumer 2 consumed Message 14 from position 3
Buffer: 0 0 0 0 0
Producer 2 produced Message 15 at position 4
Buffer: 0 0 0 0 15
```

=== Code Execution Successful ===

Activate Windows

Task 2 — Dining Philosophers with Deadlock Avoidance (Waiter Solution)

Scenario

Your restaurant introduces a rule:

Only 4 philosophers may attempt to pick forks at the same time.

This prevents deadlock.

Student Task

- Add an extra semaphore called waiter initialized to 4
- Before picking forks, each philosopher must:
sem_wait(&waiter);
- After eating, philosopher must:
sem_post(&waiter);
- Implement and demonstrate that deadlock no longer occurs.

Code:

```
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <semaphore.h>
5  #include <chrono>
6
7  using namespace std;
8
9  #define PHILOSOPHERS 5
10
11  mutex forks[PHILOSOPHERS];|
12
13  sem_t waiter;
14
15  // Philosopher Function
16  void philosopher(int id) {
17
18      int left = id;
18      int left = id;
19      int right = (id + 1) % PHILOSOPHERS;
20
21      for(int i = 0; i < 3; i++) {
22
23          cout << "Philosopher " << id
24          << " is thinking..." << endl;
25
26          this_thread::sleep_for(chrono::seconds(1));
27
28          // Waiter allows only 4 philosophers
29          sem_wait(&waiter);
30
31          // Pick forks
32          forks[left].lock();
33          cout << "Philosopher " << id
34          << " picked LEFT fork " << left << endl;
35
```

```
36     forks[right].lock();
37     cout << "Philosopher " << id
38     << " picked RIGHT fork " << right << endl;
39
40     // Eating
41     cout << "Philosopher " << id
42     << " is eating..." << endl;
43
44     this_thread::sleep_for(chrono::seconds(1));
45
46     // Put forks down
47     forks[left].unlock();
48     forks[right].unlock();
49
50     cout << "Philosopher " << id
51     << " released forks." << endl;
```

```
52
53     sem_post(&waiter);
```

```
54 }
55 }
```

```
56 }
```

```
57
```

```
58 int main() {
```

```
59
```

```
60     thread philosophers[PHILOSOPHERS];
```

```
61
```

```
62     // Initialize waiter semaphore to 4
```

```
63     sem_init(&waiter, 0, 4);
```

```
64
```

```
65     // Create philosopher threads
```

```
66     for(int i = 0; i < PHILOSOPHERS; i++) {
```

```
67
```

```
68         philosophers[i] = thread(philosopher, i);
```

```
69     }
```

```
70
```

```
71     // Join threads
```

```
72     for(int i = 0; i < PHILOSOPHERS; i++) {
```

```
73
```

```
74         philosophers[i].join();
```

```
75     }
```

```
76
```

```
77     sem_destroy(&waiter);
```

```
78
```

```
79     return 0;
```

```
80 }
```

Output:

```
Philosopher 0 is thinking...Philosopher
Philosopher 3 is thinking...
Philosopher 4 is thinking...
Philosopher 2 is thinking...
1 is thinking...
Philosopher Philosopher 0 picked LEFT fork 0
Philosopher 0 picked RIGHT fork 1
Philosopher 0 is eating...
3 picked LEFT fork 3
Philosopher 3 picked RIGHT fork 4
Philosopher 3 is eating...
Philosopher 2 picked LEFT fork 2
Philosopher 0 released forks.
Philosopher 0 is thinking...
Philosopher 3 released forks.
Philosopher 3 is thinking...
Philosopher 1 picked LEFT fork 1
Philosopher 4 picked LEFT fork 4
```

Activate Wi

```
Philosopher 4 picked LEFT fork 4
Philosopher 4 picked RIGHT fork 0
Philosopher 4 is eating...
Philosopher 2 picked RIGHT fork 3
Philosopher 2 is eating...
Philosopher 4 released forks.
Philosopher Philosopher 4 is thinking...
Philosopher 1 picked RIGHT fork 2
Philosopher 1 is eating...
Philosopher 0 picked LEFT fork 0
2 released forks.
Philosopher 2 is thinking...
Philosopher 3 picked LEFT fork 3
Philosopher 3 picked RIGHT fork 4
Philosopher 3 is eating...
Philosopher 1 released forks.
Philosopher 1 is thinking...
Philosopher 2 picked LEFT fork 2
```

Activat

```
Philosopher 2 picked LEFT fork 2
Philosopher 0 picked RIGHT fork 1
Philosopher 0 is eating...
Philosopher 3 released forks.
Philosopher 3 is thinking...
Philosopher 2 picked RIGHT fork 3
Philosopher 2 is eating...
Philosopher 4 picked LEFT fork 4
Philosopher 0 released forks.
Philosopher 0 is thinking...
1 picked LEFT fork 1
Philosopher 4 picked RIGHT fork 0
Philosopher 4 is eating...
Philosopher 2 released forks.
Philosopher 2 is thinking...
Philosopher 1 picked RIGHT fork 2
Philosopher 1 is eating...
Philosopher 3 picked LEFT fork 3
```

Activate

```
Philosopher 4 picked RIGHT fork 0
Philosopher 4 is eating...
Philosopher 2 released forks.
Philosopher 2 is thinking...
Philosopher 1 picked RIGHT fork 2
Philosopher 1 is eating...
Philosopher 3 picked LEFT fork 3
Philosopher 4 released forks.
Philosopher 4 is thinking...
Philosopher 0 picked LEFT fork 0
Philosopher 3 picked RIGHT fork 4
Philosopher 3 is eating...
Philosopher 1 released forks.
Philosopher 1 is thinking...
Philosopher 2 picked LEFT fork 0
Philosopher 2 picked RIGHT fork 1
Philosopher 0 is eating...
Philosopher 3 released forks
```

Activate Windows

```
1
Philosopher 0 is eating...
Philosopher 3 released forks.
Philosopher 4 picked LEFT fork 4
2 picked RIGHT fork 3
Philosopher 2 is eating...
Philosopher 0 released forks.
Philosopher 4 picked RIGHT fork 0
Philosopher 4 is eating...
Philosopher 1 picked LEFT fork 1
Philosopher 2 released forks.
Philosopher 4 released forks.
Philosopher 1 picked RIGHT fork 2
Philosopher 1 is eating...
Philosopher 1 released forks.
```

```
=== Code Execution Successful ===
```

Acti

Task 3 — Priority Consumer Simulation (Emergency Task Handler)

Scenario

A hospital monitoring system generates health sensor data:

- Normal consumers process routine data
- Priority consumer handles emergency alerts first

Student Task

Modify the Producer–Consumer solution such that:

- If priority consumer thread is waiting:
 - All normal consumers must wait
- Priority consumer is always served first
- Use an additional semaphore `priority_lock`
- Demonstrate how emergency items are consumed faster

Code:

```
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <condition_variable>
5  using namespace std;
6  int buffer = 0;
7  bool full = false;
8  mutex mtx;
9  condition_variable cv;
10 bool priority_waiting = false;
11 // Producer
12 void producer() {
13     for(int i = 1; i <= 10; i++) {
14         unique_lock<mutex> lock(mtx);
15         cv.wait(lock, [] {
16             return !full;
17         });
18         buffer = i;
19         full = true;
20         cout << "Producer produced: " << i << endl;
21         lock.unlock();
22         cv.notify_all();
23     }
24 // Normal Consumer
25 void normalConsumer(int id) {
26     for(int i = 0; i < 3; i++) {
27         unique_lock<mutex> lock(mtx);
28         cv.wait(lock, [] {
29             return full && !priority_waiting;
30         });
31         cout << "Normal Consumer "
32             << id
33             << " consumed: "
34             << buffer << endl;
35         full = false;
```

```

35     full = false;
36     lock.unlock();
37     cv.notify_all();
38     }}
39 // Priority Consumer
40 void priorityConsumer() {
41     for(int i = 0; i < 4; i++) {
42         unique_lock<mutex> lock(mtx);
43         priority_waiting = true;
44         cv.wait(lock, [] {
45             return full;
46         });
47         cout << "*** PRIORITY consumed: "
48             << buffer << " ***" << endl;
49         full = false;
50         priority_waiting = false;
51         lock.unlock();
52         cv.notify_all();
52         cv.notify_all();
53     }}
54 int main() {
55     thread p(producer);
56     thread n1(normalConsumer, 1);
57     thread n2(normalConsumer, 2);
58     thread pr(priorityConsumer);
59     p.join();
60     n1.join();
61     n2.join();
62     pr.join();
63     return 0;
64 }

```

Output:

```
Producer produced: 1
Normal Consumer 1 consumed: 1
Producer produced: 2
*** PRIORITY consumed: 2 ***
Producer produced: 3
*** PRIORITY consumed: 3 ***
Producer produced: 4
*** PRIORITY consumed: 4 ***
Producer produced: 5
*** PRIORITY consumed: 5 ***
Producer produced: 6
Normal Consumer 2 consumed: 6
Producer produced: 7
Normal Consumer 1 consumed: 7
Producer produced: 8
Normal Consumer 2 consumed: 8
Producer produced: 9
Normal Consumer 1 consumed: 9
*** PRIORITY consumed: 5 ***
Producer produced: 4
*** PRIORITY consumed: 4 ***
Producer produced: 5
*** PRIORITY consumed: 5 ***
Producer produced: 6
Normal Consumer 2 consumed: 6
Producer produced: 7
Normal Consumer 1 consumed: 7
Producer produced: 8
Normal Consumer 2 consumed: 8
Producer produced: 9
Normal Consumer 1 consumed: 9
Producer produced: 10
Normal Consumer 2 consumed: 10
```

```
=== Code Execution Successful ===
```

Task 4 — Smart Dining Philosophers (Time-Sharing & Starvation Prevention)

Scenario

A smart dining system ensures:

- No philosopher eats twice until all others have eaten once.

Student Task

Implement a solution with:

- An array `times_eaten[5]`
- A rule:
 - A philosopher can only eat if their `times_eaten` value is not more than 1 above the minimum of others
- Use mutex to protect access to `times_eaten`

Code:

```
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <chrono>
5
6  using namespace std;
7
8  #define N 5
9
10 int times_eaten[N] = {0};
11 mutex mtx;
12
13 int getMin() {
14
15     int mn = times_eaten[0];
16
17     for(int i = 1; i < N; i++)
18         if(times_eaten[i] < mn)
```

```

18         if(times_eaten[i] < mn)
19             mn = times_eaten[i];
20
21     return mn;
22 }
23
24 void philosopher(int id) {
25
26     for(int i = 0; i < 3; i++) {
27
28         this_thread::sleep_for(chrono::milliseconds(200));
29
30         mtx.lock();
31
32         int mn = getMin();
33
34         if(times_eaten[id] <= mn + 1) {
35

```

```

35
36         times_eaten[id]++;
37
38         cout << "Philosopher " << id
39             << " is eating | count = "
40             << times_eaten[id] << endl;
41     }
42
43     else {
44         cout << "Philosopher " << id
45             << " waiting..." << endl;
46     }
47
48     mtx.unlock();
49
50     this_thread::sleep_for(chrono::milliseconds(300));
51 }
52 }

```

```

52 }
53
54 int main() {
55
56     thread t[N];
57
58     for(int i = 0; i < N; i++)
59         t[i] = thread(philosopher, i);
60
61     for(int i = 0; i < N; i++)
62         t[i].join();
63
64     return 0;
65 }

```

Output:

```

Philosopher 0 is eating | count = 1
Philosopher 2 is eating | count = 1
Philosopher 1 is eating | count = 1
Philosopher 3 is eating | count = 1
Philosopher 4 is eating | count = 1
Philosopher 0 is eating | count = 2
Philosopher 2 is eating | count = 2
Philosopher 1 is eating | count = 2
Philosopher 4 is eating | count = 2
Philosopher 3 is eating | count = 2
Philosopher 0 is eating | count = 3
Philosopher 2 is eating | count = 3
Philosopher 4 is eating | count = 3
Philosopher 1 is eating | count = 3
Philosopher 3 is eating | count = 3

```

=== Code Execution Successful ===

Activate
Gate Set

Lab Session 10

Objective: Understanding and Implementing Bankers algorithm.

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

Write a C program to simulate disk scheduling algorithms

- FCFS
- SCAN
- C-SCAN

Required Equipment / tools:

1. Dev C++

TASK1:

DESCRIPTION

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a deadlock. Deadlock avoidance is one of the techniques for handling deadlocks. This approach requires that the operating system be given in advance additional information concerning which resources a process will request and use during its lifetime. With this additional knowledge, it can decide for each request whether or not the process should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

Banker's algorithm is a deadlock avoidance algorithm that is applicable to a system with multiple instances of each resource type.

PROGRAM

```
#include<stdio.h> struct file
{
int all[10]; int max[10]; int need[10]; int flag;
};
void main()
{
struct file f[10]; int fl;
int i, j, k, p, b, n, r, g, cnt=0, id, newr; int avail[10],seq[10];
clrscr();
```

```

printf("Enter number of processes -- "); scanf("%d",&n);
printf("Enter number of resources -- "); scanf("%d",&r);
for(i=0;i<n;i++)
{
printf("Enter details for P%d",i); printf("\nEnter allocation\t -- \t"); for(j=0;j<r;j++)
scanf("%d",&f[i].all[j]); printf("Enter Max\t\t -- \t"); for(j=0;j<r;j++)
scanf("%d",&f[i].max[j]);
f[i].flag=0;
}
printf("\nEnter Available Resources\t -- \t"); for(i=0;i<r;i++)
scanf("%d",&avail[i]);

printf("\nEnter New Request Details -- "); printf("\nEnter pid \t -- \t"); scanf("%d",&id);
printf("Enter Request for Resources \t -- \t"); for(i=0;i<n;i++)
{
scanf("%d",&newr); f[id].all[i] += newr;

avail[i]=avail[i] - newr;
}

for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
f[i].need[j]=f[i].max[j]-f[i].all[j];if(f[i].need[j]<0)
f[i].need[j]=0;
}
}
cnt=0;
fl=0;
while(cnt!=n)
{
g=0;
for(j=0;j<n;j++)
{
if(f[j].flag==0)
{
b=0;
for(p=0;p<r;p++)
{
if(avail[p]>=f[j].need[p])
b=b+1;
else
b=b-1;
}
if(b==r)
{
printf("\nP%d is visited",j); seq[fl++]=j;
f[j].flag=1;
for(k=0;k<r;k++)
avail[k]=avail[k]+f[j].all[k];
cnt=cnt+1;
printf("");
for(k=0;k<r;k++)
printf("%3d",avail[k]);

```

```

printf(" ");
g=1;
}
}
}
if(g==0)
{
printf("\n REQUEST NOT GRANTED -- DEADLOCK OCCURRED"); printf("\n SYSTEM IS IN
UNSAFE STATE");
goto y;
}
}
printf("\nSYSTEM IS IN SAFE STATE"); printf("\nThe Safe Sequence is -- ("); for(i=0;i<fl;i++)
    printf("P%d ",seq[i]); printf(")");
y:printf("\nProcess\t\tAllocation\t\tMax\t\t\tNeed\n");
for(i=0;i<n;i++)
{
printf("P%d\t",i);
for(j=0;j<r;j++)

printf("%6d",f[i].all[j]);

for(j=0;j<r;j++)

printf("%6d",f[i].max[j]);

for(j=0;j<r;j++)

printf("%6d",f[i].need[j]);

printf("\n");

}

getch();

}

```

INPUT

```

Enter number of processes    --    5

Enter number of resources    --    3

Enter details for P0

Enter allocation             --    0          1    0

Enter Max                    --          7          5          3

Enter details for P1

Enter allocation             --    2          0          0

Enter Max                    --    3          2          2

```

Enter details for P2

Enter allocation -- 3 0 2

Enter Max -- 9 0 2

Enter details for P3

Enter allocation -- 2 1 1

Enter Max -- 2 2 2

Enter details for P4

Enter allocation -- 0 0 2

Enter Max -- 4 3 3

Enter Available
Resources -- 33 2

Enter New Request
Details --

Enter pid -- 1

Enter Request for
Resources --1 0 2

OUTPUT

P1 is visited(5 3 2)

P3 is visited(7 4 3)

P4 is visited(7 4 5)

P0 is visited(7 5 5)

P2 is visited(10 5 7)

SYSTEM IS IN SAFE
STATE

The Safe Sequence is -- (P1 P3 P4 P0
P2)

Process	Allocation			Max			Need	
P0	0	1	0	7	5	3	7	43
P1	3	0	2	3	2	2	0	20
P2	3	0	2	9	0	2	6	00
P3	2	1	1	2	2	2	0	11

TASK2:**DESCRIPTION**

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and large disk bandwidth. Both the access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced which is called as disk scheduling. The simplest form of disk scheduling is, of course, the first-come, first-served (FCFS) algorithm. This algorithm is intrinsically fair, but it generally does not provide the fastest service. In the SCAN algorithm, the disk arm starts at one end, and moves towards the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk. At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk. C-SCAN is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

PROGRAM**FCFS DISK SCHEDULING ALGORITHM**

```
#include<stdio.h>
```

```
void main() {
```

```
    int t[20], n, i, j, tohm[20], tot = 0;
```

```
    float avhm;
```

```
    // Input the number of tracks
```

```
    printf("Enter the number of tracks: ");
```

```
    scanf("%d", &n);
```

```
    // Input the tracks to be traversed
```

```
    printf("Enter the tracks to be traversed: ");
```

```
    for(i = 0; i < n; i++) {
```

```
        scanf("%d", &t[i]);
```

```
}
```

```
// Calculate the difference between the tracks (header movements)
```

```
for(i = 1; i < n; i++) {
```

```
    tohm[i] = t[i] - t[i-1];
```

```
    if(tohm[i] < 0) {
```

```
        tohm[i] = -tohm[i]; // Take the absolute value of the difference
```

```
    }
```

```
}
```

```
// Calculate the total header movement
```

```
for(i = 1; i < n; i++) {
```

```
    tot += tohm[i];
```

```
}
```

```
// Calculate the average header movement
```

```
avhm = (float)tot / (n - 1);
```

```
// Output the results
```

```
printf("\nTracks traversed\tDifference between tracks\n");
```

```
for(i = 0; i < n; i++) {
```

```
    if(i == 0) {
```

```
        printf("%d\t\t0\n", t[i]); // First track has no movement, so difference is 0
```

```
    } else {
```

```
        printf("%d\t\t%d\n", t[i], tohm[i]);
```

```
    }
```

```
}
```

```
printf("\nAverage head movements: %.2f", avhm);
```

```
}
```

INPUT

Enter no.of tracks:9

Enter track position:55 58 60 70 18 90 150 160 184

OUTPUT

Tracks traversed	Difference between tracks
55	45
58	3
60	2
70	10
18	52
90	72

SCAN DISK SCHEDULING ALGORITHM

```
#include<stdio.h>
```

```
void main() {
```

```
int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum = 0;
```

```
// Input the number of tracks to be traversed
```

```
printf("Enter the number of tracks to be traversed: ");
```

```
scanf("%d", &n);
```

```
// Input the position of the head
```

```
printf("Enter the position of head: ");
```

```
scanf("%d", &h);
```

```
// Initialize the tracks array
```

```
t[0] = 0; // Assuming the 0 track is part of the request
```

```
t[1] = h; // Head starts at the given position
```

```
// Input the tracks to be traversed
```

```
printf("Enter the tracks: ");
```

```
for(i = 2; i < n + 2; i++) {
```

```

scanf("%d", &t[i]);
}
// Sort the tracks array in ascending order
for(i = 0; i < n + 2; i++) {
    for(j = 0; j < (n + 2) - i - 1; j++) {
        if(t[j] > t[j + 1]) {
            temp = t[j];
            t[j] = t[j + 1];
            t[j + 1] = temp;
        }
    }
}
// Find the position of the head in the sorted tracks array
for(i = 0; i < n + 2; i++) {
    if(t[i] == h) {
        j = i; // j points to the head's position
        k = i; // k will be used later for the second half of the traversal
    }
}
p = 0;
// First, serve tracks from the head's position towards 0
while(t[j] != 0) {
    atr[p] = t[j];
    j--;
    p++;
}
atr[p] = t[j]; // Add the 0 track
// Then, serve the remaining tracks towards the maximum track
for(p = k + 1; p < n + 2; p++, k++) {

```

```

atr[p] = t[k + 1];
}
// Calculate total header movements
for(j = 0; j < n + 1; j++) {
if(atr[j] > atr[j + 1]) {
d[j] = atr[j] - atr[j + 1];
} else {
d[j] = atr[j + 1] - atr[j];
}
sum += d[j];
}
// Output the average header movements
printf("\nAverage head movements: %.2f\n", (float)sum / n);
}

```

INPUT

Enter no.of
tracks:9

Enter track
position:55 58 60 70 18 90 150 160 184

C-SCAN DISK SCHEDULING ALGORITHM

```
#include <stdio.h>
```

```
void main() {
```

```

int t[20], d[20], h, i, j, n, temp, k, atr[20], tot, p, sum = 0;

// Input the number of tracks to be traversed
printf("Enter the number of tracks to be traversed: ");
scanf("%d", &n);

// Input the position of the head
printf("Enter the position of head: ");
scanf("%d", &h);

// Input the total number of tracks (maximum track number)
printf("Enter the total number of tracks: ");
scanf("%d", &tot);

// Initialize the tracks array
t[0] = 0; // 0 is the starting track
t[1] = h; // The initial head position

// Input the tracks to be traversed
printf("Enter the tracks: ");
for(i = 2; i < n + 2; i++) {
    scanf("%d", &t[i]);
}

// Sort the tracks array in ascending order
for(i = 0; i < n + 2; i++) {
    for(j = 0; j < (n + 2) - i - 1; j++) {
        if(t[j] > t[j + 1]) {
            temp = t[j];
            t[j] = t[j + 1];
            t[j + 1] = temp;
        }
    }
}

// Find the head position in the sorted tracks array

```

```

for(i = 0; i < n + 2; i++) {
    if(t[i] == h) {
        j = i; // j points to the position of the head
        break;
    }
}

// First, serve tracks from the head's position towards the leftmost track (0)
p = 0;
while(j >= 0 && t[j] != 0) {
    atr[p] = t[j];
    j--;
    p++;
}
atr[p] = t[j]; // Add the 0 track at the end of the leftward traversal
p++;

// Then, serve the remaining tracks from the head's position towards the highest track
for(i = j + 1; i < n + 2; i++) {
    atr[p] = t[i];
    p++;
}

// Calculate total header movements
for(j = 0; j < p - 1; j++) {
    if(atr[j] > atr[j + 1]) {
        d[j] = atr[j] - atr[j + 1];
    } else {
        d[j] = atr[j + 1] - atr[j];
    }
    sum += d[j];
}

```

```
// Output the total and average header movements
printf("Total head movements: %d\n", sum);
printf("Average head movements: %.2f\n", (float)sum / (n));
}
```

INPUT

Enter the track position :

55 58 60 70 18 90 150 160 184

Enter starting position :

100

Task 1: Construct Banker's algorithm for deadlock avoidance in resource allocation systems.

Enter Max Matrix:

Enter Available Resources:

System is in a SAFE state.

Safe Sequence: P1 P3 P4 P0 P2

Task 2: Apply deadlock detection using Banker's algorithm for process management.

```
Enter number of processes: 2
Enter number of resource types: 2
Enter Allocation Matrix:
12
2
3
23
Enter Max Matrix:
3
2
1
2
```

```
Enter Available Resources: 2
2
```

```
SYSTEM IS IN SAFE STATE
Safe Sequence: P0 -> P1
```

```
=== Code Execution Successful ===
```

Task 3: Construct a safe sequence for resource allocation with Banker's algorithm.

The SAFE Sequence is:

P1 -> P3 -> P4 -> P0 -> P2

Lab Session 11

Objective: Understanding the difference between MVT and MFT. Implementing MFT, MVT, Best fit, Worst fit and First Fit Memory Management Techniques.

Write a C program to simulate the MVT and MFT memory management techniques.

Write a C program to simulate the following contiguous memory allocation techniques

- Worst-fit
- Best-fit
- First-fit

Write a C program to simulate paging technique of memory management.

Required Equipment / tools:

1. Dev C++

TASK1:

DESCRIPTION

MFT (Multiprogramming with a Fixed number of Tasks) is one of the old memory management techniques in which the memory is partitioned into fixed size partitions and each job is assigned to a partition. The memory assigned to a partition does not change. MVT (Multiprogramming with a Variable number of Tasks) is the memory management technique in which each job gets just the amount of memory it needs. That is, the partitioning of memory is dynamic and changes as jobs enter and leave the system. MVT is a more "efficient" user of resources. MFT suffers with the problem of internal fragmentation and MVT suffers with external fragmentation.

PROGRAM

MFT MEMORY MANAGEMENT TECHNIQUE

```
#include <stdio.h>
```

```
#include <stdlib.h> // For system("cls") or system("clear")
```

```

int main() {

    int ms, bs, nob, ef, n, mp[10], tif = 0; // Total memory size, block size, etc.

    int i, p = 0; // Loop variables and process count


    printf("Enter the total memory available (in Bytes) -- ");

    scanf("%d", &ms);


    printf("Enter the block size (in Bytes) -- ");

    scanf("%d", &bs);


    if (bs <= 0 || ms <= 0 || bs > ms) {

        printf("Invalid input! Block size must be positive and less than or equal to memory size.\n");

        return 1;

    }


    nob = ms / bs;

    ef = ms - nob * bs;


    printf("\nEnter the number of processes -- ");

    scanf("%d", &n);


    for (i = 0; i < n; i++) {

        printf("Enter memory required for process %d (in Bytes)-- ", i + 1);

        scanf("%d", &mp[i]);

    }

```

```
printf("\nNo. of Blocks available in memory -- %d", nob);

printf("\n\nPROCESS\tMEMORY\t\t\t\t\tREQUIRED\tALLOCATED\tINTERNAL  
FRAGMENTATION");

for (i = 0; i < n && p < nob; i++) {

    printf("\n %d\t\t%d", i + 1, mp[i]);

    if (mp[i] > bs) {

        printf("\t\tNO\t\t---");

    } else {

        printf("\t\tYES\t\t%d", bs - mp[i]);

        tif += bs - mp[i];

        p++;

    }

}


if (i < n) {

    printf("\nMemory is Full, Remaining Processes cannot be accommodated.");

    for (; i < n; i++) {

        printf("\n %d\t\t%d\t\t\t\t\tNO\t\t\t---", i + 1, mp[i]);

    }

}


printf("\n\nTotal Internal Fragmentation is %d", tif);

printf("\nTotal External Fragmentation is %d", ef);

return 0;
```

INPUT

Enter the total memory available (in Bytes) --
Enter the block size (in Bytes)-- 300
Enter the number of processes – 5
Enter memory required for process 1 (in Bytes) --
Enter memory required for process 2 (in Bytes) --
Enter memory required for process 3 (in Bytes) --
Enter memory required for process 4 (in Bytes) --
Enter memory required for process 5 (in Bytes) --
No. of Blocks available in memory -- 3

OUTPUT

PROCESS	MEMORY REQUIRED
1	275
2	400
3	290
4	293

Memory is Full, Remaining Processes cannot be accommodated

Total Internal Fragmentation is	42
Total External Fragmentation is	100

MVT MEMORY MANAGEMENT TECHNIQUE

```
#include<stdio.h>

main()
{
    int ms, mp[10], i, temp, n = 0;
    char ch = 'y';
    printf("\nEnter the total memory available (in Bytes) -- ");
    scanf("%d", &ms);
    temp = ms; // Temporary variable to track available memory
    for(i = 0; ch == 'y'; i++, n++)
```

```

{
printf("\nEnter memory required for process %d (in Bytes) -- ", i + 1);
scanf("%d", &mp[i]);
if(mp[i] <= temp)
{
printf("\nMemory is allocated for Process %d ", i + 1);
temp = temp - mp[i]; // Reduce the available memory by allocated memory
}
else
{
printf("\nMemory is Full");
break; // Exit if memory is full
}
printf("\nDo you want to continue(y/n) -- ");
scanf(" %c", &ch); // Adding space before %c to capture the newline character
}
printf("\n\nTotal Memory Available -- %d", ms);
printf("\n\n\tPROCESS\t\t MEMORY ALLOCATED ");
for(i = 0; i < n; i++)
printf("\n \t%d\t\t\t%d", i + 1, mp[i]);
// Output the total memory allocated and external fragmentation
printf("\n\nTotal Memory Allocated is %d", ms - temp);
printf("\nTotal External Fragmentation is %d", temp);
return 0;
}

```

INPUT

Enter the total memory available (in Bytes) -- 1000

Enter memory required for process 1 (in Bytes) -- 400

Memory is allocated for
Process 1

Do you want to continue(y/n) -- y

Enter memory required for process 2 (in
Bytes) -- 275

Memory is allocated for
Process 2

Do you want to continue(y/n) -- y

Enter memory required for process 3 (in
Bytes) -- 550

OUTPUT

Memory is Full

Total Memory Available -- 1000

PROCESS	MEMORY ALLOCATED
---------	------------------

1	400
---	-----

2	275
---	-----

Total Memory Allocated is 675

Total External Fragmentation is 325

TASK2:

DESCRIPTION

One of the simplest methods for memory allocation is to divide memory into several fixed-sized partitions. Each partition may contain exactly one process. In this multiple-partition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process. The operating system keeps a table indicating which parts of memory are available and which are occupied. Finally, when a process arrives and needs memory, a memory section large enough for this process is provided. When it is time to load or swap a process into main memory, and if there is more than one free block of memory of sufficient size, then the operating system must decide which free block to allocate. Best-fit strategy chooses the block that is closest in size to the request. First-fit chooses the first available block that is large enough. Worst-fit chooses the largest available block.

PROGRAM

WORST-FIT

```
#include<stdio.h>

#define max 25

void main()

{

    int frag[max], b[max], f[max], i, j, nb, nf, temp;

    static int bf[max], ff[max];


    // Input the number of blocks and files

    printf("\n\tMemory Management Scheme - Worst Fit\n");

    printf("\nEnter the number of blocks: ");

    scanf("%d", &nb);

    printf("Enter the number of files: ");

    scanf("%d", &nf);


    // Input the sizes of the blocks

    printf("\nEnter the size of the blocks:\n");

    for(i = 1; i <= nb; i++)

    {

        printf("Block %d: ", i);

        scanf("%d", &b[i]);

    }


    // Input the sizes of the files
```

```

printf("Enter the size of the files:\n");

for(i = 1; i <= nf; i++)
{
    printf("File %d: ", i);

    scanf("%d", &f[i]);
}

// Initialize the block allocation array

for(i = 1; i <= nf; i++)
{
    int worstIdx = -1;

    int maxFrag = -1;

    // Find the largest block that can accommodate the current file (Worst-Fit strategy)

    for(j = 1; j <= nb; j++)
    {
        if(bf[j] != 1) // Check if the block is not already allocated
        {
            temp = b[j] - f[i];

            if(temp >= 0 && temp > maxFrag) // Check if block can accommodate and if it's the
largest
            {
                maxFrag = temp;

                worstIdx = j;
            }
        }
    }
}

```

```

    }

    if(worstIdx != -1) // If a block is found to allocate
    {
        ff[i] = worstIdx; // Store the block number allocated

        frag[i] = maxFrag; // Store the fragmentation

        bf[worstIdx] = 1; // Mark the block as allocated
    }

    else

    {
        ff[i] = -1; // No block found for this file

        frag[i] = -1; // No fragmentation as the file is not allocated
    }
}

// Output the results

printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement\n");

for(i = 1; i <= nf; i++)
{
    if(ff[i] != -1)

        printf("%d\t%d\t%d\t%d\t%d\n", i, f[i], ff[i], b[ff[i]], frag[i]);

    else

        printf("%d\t%d\t\tNot Allocated\t\t---\t\t---\n", i, f[i]);
}

```

```
return 0;
```

```
}
```

INPUT

Enter the number of blocks: 3 Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	1	5	4
2	4	3	7	3

BEST-FIT

```
#include<stdio.h>
```

```
#define max 25
```

```
void main()
```

```
{
```

```
int frag[max], b[max], f[max], i, j, nb, nf, temp, lowest;
```

```
static int bf[max], ff[max]; // bf[]: Block allocated, ff[]: File allocated to block
```

```
printf("\nEnter the number of blocks: ");
```

```
scanf("%d", &nb);
```

```
printf("Enter the number of files: ");
```

```
scanf("%d", &nf);
```

```
// Input block sizes
```

```
printf("\nEnter the size of the blocks:\n");
```

```

for(i = 1; i <= nb; i++)
{
printf("Block %d: ", i);
scanf("%d", &b[i]);
}

// Input file sizes
printf("Enter the size of the files:\n");
for(i = 1; i <= nf; i++)
{
printf("File %d: ", i);
scanf("%d", &f[i]);
}

// Initialize block allocation and fragmentation
for(i = 1; i <= nf; i++)
{
lowest = 10000; // Set a high initial value for the lowest fragmentation
for(j = 1; j <= nb; j++)
{
if(bf[j] != 1) // Check if the block is already allocated
{
temp = b[j] - f[i]; // Calculate the remaining space after allocation
if(temp >= 0 && temp < lowest) // Check if this block is the best fit
{
ff[i] = j; // Assign the block number
lowest = temp; // Update the lowest fragmentation
}
}
}

// If a block is allocated, update the block as allocated and store fragmentation

```

```

if(ff[i] != 0)
{
    frag[i] = lowest;
    bf[ff[i]] = 1; // Mark the block as allocated
}
else
{
    frag[i] = -1; // If no block is found, set fragmentation as -1
}
}

// Print results

printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");

for(i = 1; i <= nf; i++)
{
    if(ff[i] != 0) // If the file is allocated to a block
    {
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]);
    }
    else // If the file is not allocated
    {
        printf("\n%d\t\t%d\t\tNot Allocated\t---\t---", i, f[i]);
    }
}

return;
}

```

INPUT

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	2	2	1
2	4	1	5	1

FIRST-FIT

```
#include<stdio.h>

#define max 25

void main()
{
    int frag[max], b[max], f[max], i, j, nb, nf, temp, highest = 0;
    static int bf[max], ff[max]; // bf[]: Block allocated, ff[]: File allocated to block
    printf("\n\tMemory Management Scheme - Worst Fit");
    printf("\n\tEnter the number of blocks: ");
    scanf("%d", &nb);
    printf("Enter the number of files: ");
    scanf("%d", &nf);
    // Input block sizes
    printf("\n\tEnter the size of the blocks:\n");
    for(i = 1; i <= nb; i++) {
        printf("Block %d: ", i);
        scanf("%d", &b[i]);
    }
```

```

// Input file sizes
printf("Enter the size of the files:\n");

for(i = 1; i <= nf; i++) {
    printf("File %d: ", i);
    scanf("%d", &f[i]);
}

// Memory allocation using Worst-Fit strategy
for(i = 1; i <= nf; i++) {
    highest = -1; // Reset highest at the start of each file allocation
    for(j = 1; j <= nb; j++) {
        if(bf[j] != 1) { // Check if the block is not already allocated
            temp = b[j] - f[i]; // Calculate the remaining space after file allocation
            if(temp >= 0 && temp > highest) { // Find the block with the maximum remaining space
                ff[i] = j; // Allocate the file to this block
                highest = temp; // Update the highest fragmentation
            }
        }
    }

    if(ff[i] != 0) { // If file is allocated to a block
        frag[i] = highest; // Store the fragmentation value
        bf[ff[i]] = 1; // Mark the block as allocated
    } else {
        frag[i] = -1; // If no block is found, mark it as not allocated
    }
}

// Print allocation results
printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");

for(i = 1; i <= nf; i++) {
    if(ff[i] != 0) {

```

```

printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]);
} else {
printf("\n%d\t\t%d\t\tNot Allocated\t---\t\t---", i, f[i]);
}
}
return;
}

```

INPUT

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	3	7	6
2	4	1	5	1

Task 1:

Allocate files to blocks using Best Fit, minimizing fragmentation by choosing the block with least leftover space.

```
--- Best Fit File Allocation ---
Enter number of blocks: 2
Enter number of files: 2
Enter size of blocks:
22
12
Enter size of files:
10
11

File No  File Size  Block No  Remaining Block Space
1      10         2         2
2      11         1        11
```

Code:

```
1  #include<stdio.h>
2  #define max 25
3
4  int main()
5  {
6      int frag[max], b[max], f[max], i, j, nb, nf, temp, lowest;
7      static int bf[max], ff[max];
8
9      printf("Enter the number of blocks: ");
10     scanf("%d", &nb);
11     printf("Enter the number of files: ");
12     scanf("%d", &nf);
13
14     printf("Enter the size of the blocks:\n");
15     for(i = 1; i <= nb; i++)
16         scanf("%d", &b[i]);
17
18     printf("Enter the size of the files:\n");
19     for(i = 1; i <= nf; i++)
20         scanf("%d", &f[i]);
```

```

21
22     for(i = 1; i <= nf; i++)
23     {
24         lowest = 10000;
25         for(j = 1; j <= nb; j++)
26         {
27             if(bf[j] != 1)
28             {
29                 temp = b[j] - f[i];
30                 if(temp >= 0 && temp < lowest)
31                 {
32                     ff[i] = j;
33                     lowest = temp;
34                 }
35             }
36         }
37
38         if(ff[i] != 0)
39         {
40             frag[i] = lowest;

```

```

38         if(ff[i] != 0)
39         {
40             frag[i] = lowest;
41             bf[ff[i]] = 1;
42         }
43     }
44
45     printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");
46     for(i = 1; i <= nf; i++)
47         printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]],
48             frag[i]);
49     return 0;
50 }

```

Output:

```

Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of the blocks:
5
2
7
Enter the size of the files:
1
4

File No File Size   Block No   Block Size  Fragment
1       1       2       2       1
2       4       1       5       1

```

Task 2:

Compare First Fit and Worst Fit allocation strategies, then calculate and display fragmentation for both methods.

```
--- Memory Allocation Benchmark ---  
  
First Fit Results:  
File      Size      Block      Rem. Space  
F1  212 B2  176  
F2  417 B5  183  
F3  112 B2  176  
F4  426 None  -  
Total Internal Fragmentation: 535
```

```
Worst Fit Results:  
File      Size      Block      Rem. Space  
F1  212 B5  276  
F2  417 B2   83  
F3  112 B5  276  
F4  426 None  -  
Total Internal Fragmentation: 635
```

Code:

FIRST FIT:

```
1  #include<stdio.h>  
2  #define max 25  
3  int main()  
4  {  
5      int frag[max], b[max], f[max], i, j, nb, nf;  
6      static int bf[max], ff[max];  
7      printf("\nMemory Management Scheme - First Fit");  
8      printf("\nEnter the number of blocks: ");  
9      scanf("%d", &nb);  
10     printf("Enter the number of files: ");  
11     scanf("%d", &nf);  
12     // Input block sizes  
13     printf("\nEnter the size of the blocks:\n");  
14     for(i = 1; i <= nb; i++){  
15         printf("Block %d: ", i);  
16         scanf("%d", &b[i]);  
17     }  
18     // Input file sizes  
19     printf("\nEnter the size of the files:\n");  
20     for(i = 1; i <= nf; i++){  
21         printf("File %d: ", i);  
22         scanf("%d", &f[i]);  
23     }
```

```

21     scanf("%d", &f[i]);}
22     // First Fit Allocation
23     for(i = 1; i <= nf; i++)
24     {for(j = 1; j <= nb; j++) {
25         if(bf[j] != 1 && b[j] >= f[i]) {
26             ff[i] = j;
27             frag[i] = b[j] - f[i];
28             bf[j] = 1;
29             break;}}
30     if(ff[i] == 0)
31         frag[i] = -1;}
32     // Display Results
33     printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");
34     for(i = 1; i <= nf; i++){
35         if(ff[i] != 0)
36             printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",
37                 i, f[i], ff[i], b[ff[i]], frag[i]);
38         else
39             printf("\n%d\t\t%d\t\t\t\t\tNot Allocated\t---\t\t---",
40                 i, f[i]);}
41     return 0;}

```

```

38     else
39         printf("\n%d\t\t%d\t\t\t\t\tNot Allocated\t---\t\t---",
40             i, f[i]);}
41     return 0;}

```

WORST FIT:

```

1  #include<stdio.h>
2  #define max 25
3  int main()
4  {
5      int frag[max], b[max], f[max], i, j, nb, nf, temp;
6      static int bf[max], ff[max];
7      printf("Enter number of blocks: ");
8      scanf("%d",&nb);
9      printf("Enter number of files: ");
10     scanf("%d",&nf);
11     for(i=1;i<=nb;i++)
12         scanf("%d",&b[i]);
13     for(i=1;i<=nf;i++)
14         scanf("%d",&f[i]);
15     for(i=1;i<=nf;i++)
16     {
17         int highest = -1;
18         for(j=1;j<=nb;j++)
19         {
20             if(bf[j]!=1)

```

```

18     for(j=1;j<=nb;j++)
19     {
20         if(bf[j]!=1)
21         {
22             temp = b[j]-f[i];
23             if(temp>=0 && temp>highest)
24             {
25                 highest=temp;
26                 ff[i]=j;
27             }
28         }
29         frag[i]=highest;
30         bf[ff[i]]=1;
31     }
32     printf("\nFile No\tFile Size\tBlock No\tFragment");
33     for(i=1;i<=nf;i++)
34         printf("\n%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],frag[i]);
35     return 0;
36 }

```

Output:

FIRST FIT:

Output

Memory Management Scheme - First Fit

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:

Block 1: 5

Block 2: 2

Block 3: 1

Enter the size of the files:

File 1: 5

File 2: 7

File No	File Size	Block No	Block Size	Fragment
1	5	1	5	0
2	7	Not Allocated	---	---

WORST FIT:

```
Enter number of blocks: 3
Enter number of files: 2
5
2
7
1
4

File No File Size   Block No   Fragment
1       1       3       6
2       4       1       1
```

Task 3:

Compare First Fit and Best Fit allocation strategies, then calculate and display fragmentation for both methods.

```
--- ALLOCATION COMPARISON ---
File      Size      First-Fit Block Best-Fit Block
F1   212 B4         B4
F2   417 B2         B2
F3   112 B3         B3
F4   426 B5         B5

Total Internal Fragmentation (First Fit): 647
Total Internal Fragmentation (Best Fit):  433
```

Code:

FIRST FIT:

```
1  #include<stdio.h>
2  #define max 25
3  int main()
4  {
5      int frag[max], b[max], f[max], i, j, nb, nf;
6      static int bf[max], ff[max];
7      printf("\nMemory Management Scheme - First Fit");
8      printf("\nEnter the number of blocks: ");
9      scanf("%d", &nb);
10     printf("Enter the number of files: ");
11     scanf("%d", &nf);
12     // Input block sizes
13     printf("\nEnter the size of the blocks:\n");
14     for(i = 1; i <= nb; i++){
15         printf("Block %d: ", i);
16         scanf("%d", &b[i]);
17     }
18     // Input file sizes
19     printf("\nEnter the size of the files:\n");
20     for(i = 1; i <= nf; i++){
21         printf("File %d: ", i);
```

```

21     scanf("%d", &f[i]);}
22     // First Fit Allocation
23     for(i = 1; i <= nf; i++)
24     {for(j = 1; j <= nb; j++)    {
25         if(bf[j] != 1 && b[j] >= f[i])    {
26             ff[i] = j;
27             frag[i] = b[j] - f[i];
28             bf[j] = 1;
29             break;}}
30     if(ff[i] == 0)
31         frag[i] = -1;}
32     // Display Results
33     printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");
34     for(i = 1; i <= nf; i++){
35         if(ff[i] != 0)
36             printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",
37                 i, f[i], ff[i], b[ff[i]], frag[i]);
38         else
39             printf("\n%d\t\t%d\t\t\t\t\tNot Allocated\t---\t\t---",
40                 i, f[i]);}
41     return 0;}

```

```

38         else
39             printf("\n%d\t\t%d\t\t\t\t\tNot Allocated\t---\t\t---",
40                 i, f[i]);}
41     return 0;}

```

BEST FIT:

```

1  #include<stdio.h>
2  #define max 25
3
4  int main()
5  {
6      int frag[max], b[max], f[max], i, j, nb, nf, temp, lowest;
7      static int bf[max], ff[max];
8
9      printf("Enter the number of blocks: ");
10     scanf("%d", &nb);
11     printf("Enter the number of files: ");
12     scanf("%d", &nf);
13
14     printf("Enter the size of the blocks:\n");
15     for(i = 1; i <= nb; i++)
16         scanf("%d", &b[i]);
17
18     printf("Enter the size of the files:\n");
19     for(i = 1; i <= nf; i++)
20         scanf("%d", &f[i]);

```

```

21
22     for(i = 1; i <= nf; i++)
23     {
24         lowest = 10000;
25         for(j = 1; j <= nb; j++)
26         {
27             if(bf[j] != 1)
28             {
29                 temp = b[j] - f[i];
30                 if(temp >= 0 && temp < lowest)
31                 {
32                     ff[i] = j;
33                     lowest = temp;
34                 }
35             }
36         }
37
38         if(ff[i] != 0)
39         {
40             frag[i] = lowest;

```

```

38         if(ff[i] != 0)
39         {
40             frag[i] = lowest;
41             bf[ff[i]] = 1;
42         }
43     }
44
45     printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment");
46     for(i = 1; i <= nf; i++)
47         printf("\n%d\t\t\t%d\t\t\t%d\t\t\t%d\t\t\t%d", i, f[i], ff[i], b[ff[i]],
48             frag[i]);
49     return 0;
50 }

```

Output:

FIRST FIT:

```

Memory Management Scheme - First Fit
Enter the number of blocks: 3
Enter the number of files: 2

Enter the size of the blocks:
Block 1: 5
Block 2: 2
Block 3: 1

Enter the size of the files:
File 1: 5
File 2: 7

File No File Size   Block No   Block Size  Fragment
1       5         1         5          0
2       7         Not Allocated ---      ---

```

BEST FIT:

Output

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:

5

2

7

Enter the size of the files:

1

4

File No	File Size	Block No	Block Size	Fragment
---------	-----------	----------	------------	----------

1	1	2	1	
---	---	---	---	--

2	4	5	1	
---	---	---	---	--

=== Code Execution Successful ===

Lab Session 12

Objective: Understanding the difference between physical address and logical address, Resolving page table.

Write a C program to simulate the paging memory management techniques.

TASK:**Required Equipment / tools:**

1. Dev C++

TASK:**DESCRIPTION**

In computer operating systems, paging is one of the memory management schemes by which a computer stores and retrieves data from the secondary storage for use in main memory. In the paging memory-management scheme, the operating system retrieves data from secondary storage in same-size blocks called pages. Paging is a memory-management scheme that permits the physical address space a process to be noncontiguous. The basic method for implementing paging involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages. When a process is to be executed, its pages are loaded into any available memory frames from their source.

PROGRAM

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main() {
```

```
int ms, ps, nop, np, rempages, i, j, x, y, pa, offset;
```

```
int s[10], fno[10][20]; // 's' for pages required by processes, 'fno' for frame numbers.
```

```
clrscr();
```

```
// Input memory size and page size
```

```
printf("\nEnter the memory size (in bytes): ");
```

```
scanf("%d", &ms);
```

```
printf("\nEnter the page size (in bytes): ");
```

```
scanf("%d", &ps);
```

```
// Calculate number of pages available in memory
```

`nop = ms / ps;`

`printf("\nThe number of pages available in memory are: %d", nop);`

`// Input number of processes`

`printf("\nEnter the number of processes: ");`

`scanf("%d", &np);`

`rempages = nop; // Remaining pages available in memory`

`// Input process-specific page requirements`

`for (i = 1; i <= np; i++) {`

`printf("\nEnter the number of pages required for p[%d]: ", i);`

`scanf("%d", &s[i]);`

`// Check if enough memory is available`

`if (s[i] > rempages) {`

`printf("\nMemory is Full, Not Enough Pages Available");`

`break; // Exit the loop if memory is full`

`}`

`rempages = rempages - s[i]; // Update remaining pages`

`// Input the page table for the current process`

`printf("\nEnter the page table for p[%d]: ", i);`

`for (j = 0; j < s[i]; j++) {`

`scanf("%d", &fno[i][j]); // Input the frame numbers assigned to the process`

`}`

`}`

`// Input logical address (process number, page number, and offset)`

`printf("\nEnter Logical Address to find Physical Address");`

`printf("\nEnter process number, page number, and offset: ");`

`scanf("%d %d %d", &x, &y, &offset);`

`// Validate the inputs`

`if (x > np || x <= 0 || y >= s[x] || offset >= ps || y < 0 || offset < 0) {`

12Iqra University
Science

Department of Computer

printf("\nInvalid Process Number, Page Number, or Offset");

} else {

// Calculate physical address

pa = fno[x][y] * ps + offset;

printf("\nThe Physical Address is: %d", pa);

}

getch(); // Wait for user input to close the console

}

INPUTEnter the memory size --
1000

Enter the page size -- 100

The no. of pages available in memory are -- 10

Enter number of processes -- 3

Enter no. of pages required for p[1]-- 4

Enter pagetable for p[1] --- 8 6 9 5

Enter no. of pages required for p[2]-- 5

Enter pagetable for p[2] --- 1 4 5 73

Enter no. of pages required for p[3]-- 5

OUTPUT

Memory is Full

Enter Logical Address to find Physical Address

Enter process no. and pagenumber and offset -
- 2

360

The Physical Address

is -- 760

Task 1: Implement a page table management system to convert logical addresses to physical addresses in a memory allocation simulation.

Code:

```
1  #include<stdio.h>
2  int main()
3  {
4      int ms, ps, nop, np, rempages, i, j, x, y, pa, offset;
5      int s[10], fno[10][20];
6      printf("Enter the memory size (in bytes): ");
7      scanf("%d", &ms);
8      printf("Enter the page size (in bytes): ");
9      scanf("%d", &ps);
10     nop = ms / ps;
11     printf("The number of pages available in memory are: %d\n", nop);
12     printf("Enter the number of processes: ");
13     scanf("%d", &np);
14     rempages = nop;
15     for(i = 1; i <= np; i++)
16     { printf("Enter the number of pages required for P[%d]: ", i);
17       scanf("%d", &s[i]);
18       if(s[i] > rempages)
19       { printf("Memory is Full\n");
20         break;}
21
22       rempages -= s[i];
23       printf("Enter page table for P[%d]: ", i);
24       for(j = 0; j < s[i]; j++)
25       { scanf("%d", &fno[i][j]);
26         } printf("Enter process number, page number and offset: ");
27       scanf("%d%d%d", &x, &y, &offset);
28       if(x > np || y >= s[x] || offset >= ps)
29       { printf("Invalid Address\n");
30         }
31       else{
32         pa = fno[x][y] * ps + offset;
33         printf("The Physical Address is %d", pa);}
34     }
```

Output:

```
Enter the memory size (in bytes): 1000
Enter the page size (in bytes): 100
The number of pages available in memory are: 10
Enter the number of processes: 3
Enter the number of pages required for P[1]: 4
Enter page table for P[1]: 8
6
9
5
Enter the number of pages required for P[2]: 5
Enter page table for P[2]: 1
4
5
7
3
Enter the number of pages required for P[3]: 5
Memory is Full
Enter process number, page number and offset: 2
3
60
```

```
Enter process number, page number and offset: 2
3
60
The Physical Address is 760
```

```
=== Code Execution Successful ===
```

Task 2: Design and develop a program to manage memory allocation for single process with page table creation and address translation.

Code:

```
1  #include<stdio.h>
2  int main()
3  {
4      int ps, pages, offset, pageno, pa;
5      int frame[20];
6      printf("Enter page size: ");
7      scanf("%d",&ps);
8      printf("Enter number of pages required: ");
9      scanf("%d",&pages);
10     printf("Enter frame numbers:\n");
11     for(int i=0;i<pages;i++)
12         scanf("%d",&frame[i]);
13     printf("Enter page number and offset: ");
14     scanf("%d%d",&pageno,&offset);
15     pa = frame[pageno]*ps + offset;
16     printf("Physical Address = %d",pa);
17     return 0;
18 }
```

Output:

Output

```
Enter page size: 100
Enter number of pages required: 4
Enter frame numbers:
8
6
9
5
Enter page number and offset: 2
50
Physical Address = 950

=== Code Execution Successful ===
```

Task 3: Design and develop a program to manage memory allocation for multiple processes with page table creation and address translation.

Code:

```
1  #include<stdio.h>
2  int main()
3  {   int n, ps, pages[10], frame[10][20];
4      int i, j, pno, pg, offset, pa;
5      printf("Enter page size: ");
6      scanf("%d",&ps);
7      printf("Enter number of processes: ");
8      scanf("%d",&n);
9      for(i=1;i<=n;i++)
10     {   printf("Enter pages required for P[%d]: ",i);
11         scanf("%d",&pages[i]);
12         printf("Enter frame numbers for P[%d]: ",i);
13         for(j=0;j<pages[i];j++)
14             scanf("%d",&frame[i][j]);
15     }   printf("Enter process number, page number and offset: ");
16     scanf("%d%d%d",&pno,&pg,&offset);
17     pa = frame[pno][pg]*ps + offset;
18     printf("Physical Address = %d",pa);
19     return 0;
20 }
```

Output:

Output

```
Enter page size: 100
Enter number of processes: 2
Enter pages required for P[1]: 4
Enter frame numbers for P[1]: 8
6
9
5
Enter pages required for P[2]: 5
Enter frame numbers for P[2]: 1
4
5
7
3
Enter process number, page number and offset: 2
3
60
Physical Address = 760

=== Code Execution Successful ===
```

Lab Session 13

Objective: Understanding and Implementing Page Replacement Algorithms (FIFO, LRU, LFU and OPT)

Write a C program to simulate page replacement algorithms

- Optimal

Required Equipment / tools:

- Dev C++

TASK1:

DESCRIPTION

Page replacement is basic to demand paging. It completes the separation between logical memory and physical memory. With this mechanism, an enormous virtual memory can be provided for programmers on a smaller physical memory. There are many different page-replacement algorithms. Every operating system probably has its own replacement scheme. A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen. If the recent past is used as an approximation of the near future, then the page that has not been used for the longest period of time can be replaced. This approach is the Least Recently Used (LRU) algorithm. LRU replacement associates with each page the time of that page's last use. When a page must be replaced, LRU chooses the page that has not been used for the longest period of time. Least frequently used (LFU) page-replacement algorithm requires that the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count.

PROGRAM

FIFO PAGE REPLACEMENT ALGORITHM

```
#include<stdio.h>

int main() {
    int i, j, k, f, pf = 0, count = 0, rs[25], m[10], n;
    printf("\nEnter the length of reference string: ");
    scanf("%d", &n);
    printf("\nEnter the reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &rs[i]);
```

```

printf("\nEnter the number of frames: ");

scanf("%d", &f);

// Initialize the memory with -1 (empty frames)
for(i = 0; i < f; i++)
m[i] = -1;

printf("\nThe Page Replacement Process is as follows:\n");

for(i = 0; i < n; i++) {
for(k = 0; k < f; k++) {
if(m[k] == rs[i]) {
break; // Page is already in memory, no page fault
}
}
if(k == f) { // Page fault occurs
m[count++] = rs[i];
pf++;
}

// Print current frame contents
for(j = 0; j < f; j++) {
printf("\t%d", m[j]);
}

if(k == f) {
printf("\tPF No. %d", pf); // Page fault number
}

printf("\n");

if(count == f) {
count = 0; // Reset to 0 if all frames are filled
}
}

printf("\nThe number of page faults using FIFO are: %d\n", pf);

```

```
return 0;

}
```

INPUT

Enter the length of reference string – 20

Enter the reference string -- 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter no. of frames -- 3

LRU PAGE REPLACEMENT ALGORITHM

```
#include<stdio.h>
```

```
int main() {

int i, j, k, min, rs[25], m[10], count[10], flag[25], n, f, pf = 0, next = 1;

// Input: Length of reference string

printf("Enter the length of reference string: ");

scanf("%d", &n);

// Input: Reference string

printf("Enter the reference string: ");

for(i = 0; i < n; i++) {

scanf("%d", &rs[i]);

flag[i] = 0; // Mark pages as not found initially

}

// Input: Number of frames

printf("Enter the number of frames: ");

scanf("%d", &f);

// Initialize memory and count arrays

for(i = 0; i < f; i++) {

m[i] = -1; // Initially all frames are empty

count[i] = 0; // Initialize count for LRU tracking

}

printf("\n\nThe Page Replacement process is as follows:\n");

// Process each reference in the reference string
```

```

for(i = 0; i < n; i++) {
// Check if the page is already in memory
for(j = 0; j < f; j++) {
if(m[j] == rs[i]) {
flag[i] = 1; // Page is found in memory
count[j] = next; // Update the count to the most recent access
next++;
}
}
// If page is not found in memory, replace a page
if(flag[i] == 0) {
if(i < f) { // If there is space in memory, add the page
m[i] = rs[i];
count[i] = next;
next++;
} else { // Memory is full, replace the least recently used page
min = 0;
for(j = 1; j < f; j++) {
if(count[min] > count[j]) {
min = j; // Find the LRU page
}
}
m[min] = rs[i]; // Replace the LRU page with the new one
count[min] = next;
next++;
}
pf++; // Increment page fault count
}
// Print the current state of the memory

```

```

for(j = 0; j < f; j++) {
printf("%d\t", m[j]);
}
if(flag[i] == 0) {
printf("PF No. -- %d", pf); // Display the page fault number
}
printf("\n");
}
// Output the total number of page faults
printf("\nThe number of page faults using LRU are: %d\n", pf);
return 0;
}

```

INPUT

Enter the length of reference string -- 20

Enter the reference string -- 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 Enter the number of frames -- 3

LFU PAGE REPLACEMENT ALGORITHM

```

#include<stdio.h>

int main() {
int rs[50], i, j, k, m, f, cntr[20], a[20], min, pf = 0;
// Input: Number of page references
printf("\nEnter number of page references: ");
scanf("%d", &m);
// Input: Reference string
printf("\nEnter the reference string: ");
for(i = 0; i < m; i++)
scanf("%d", &rs[i]);
// Input: Number of available frames

```

```

printf("\nEnter the available number of frames: ");
scanf("%d", &f);

// Initialize counters and frames
for(i = 0; i < f; i++) {
    cntr[i] = 0; // Frequency counter for each frame
    a[i] = -1; // Initialize frames to -1 (empty)
}

printf("\nThe Page Replacement Process is as follows:\n");

// Page replacement process
for(i = 0; i < m; i++) {
    // Check if the page is already in memory
    for(j = 0; j < f; j++) {
        if(rs[i] == a[j]) {
            cntr[j]++; // Increment the frequency of the page
            break;
        }
    }

    // If the page was not found in memory
    if(j == f) {
        // Find the least frequently used page to replace
        min = 0;
        for(k = 1; k < f; k++) {
            if(cntr[k] < cntr[min])
                min = k;
        }

        // Replace the least frequently used page
        a[min] = rs[i];
        cntr[min] = 1; // Reset the frequency of the new page
        pf++; // Increment page fault count
    }
}

```

```

}
// Print the current state of the memory
for(j = 0; j < f; j++) {
printf("\t%d", a[j]);
}
// If there was a page fault, print the page fault number
if(j == f) {
printf("\tPF No. %d", pf);
}
printf("\n");
}
// Output the total number of page faults
printf("\nTotal number of page faults: %d\n", pf);
return 0;
}

```

INPUT

Enter number of page references -
-10

Enter the reference string --
1 2 3 4 5 2 5 2 5 1 4
3

Enter the available no. of frames -
- 3

TASK2:

DESCRIPTION

Optimal page replacement algorithm has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly. The basic idea is to replace the page that will not be used for the longest period of time. Use of this page-replacement algorithm guarantees the lowest possible page fault rate for a fixed number of frames. Unfortunately, the optimal page-replacement algorithm is difficult to implement, because it requires future knowledge of the reference string.

```
#include<stdio.h>
```

```

// Function to find the page to replace based on the farthest future use
int findmax(int a[], int n) {
    int max = a[0], i, k = 0;
    for(i = 1; i < n; i++) {
        if(max < a[i]) {
            max = a[i];
            k = i;
        }
    }
    return k;
}

int main() {
    int n, max, seq[30], fr[5], pos[5], flag, i, j, k, pf = 0, s;
    float pfr;

    // Input: Maximum length of the sequence
    printf("Enter maximum limit of the sequence: ");
    scanf("%d", &max);

    // Input: Reference string (page sequence)
    printf("\nEnter the sequence: ");
    for(i = 0; i < max; i++) {
        scanf("%d", &seq[i]);
    }

    // Input: Number of frames
    printf("\nEnter number of frames: ");
    scanf("%d", &n);

    // Initialize frames with -1 (indicating empty)
    for(i = 0; i < n; i++) {
        fr[i] = -1;
    }
}

```

```

// Initialize page fault count and place the first page in memory
fr[0] = seq[0];
pf++;
printf("%d\t", fr[0]);

// Fill the remaining frames with the first unique pages
int count = 1;
i = 1;
while(count < n) {
    flag = 1;
    for(j = 0; j < i; j++) {
        if(seq[i] == seq[j]) {
            flag = 0;
            break;
        }
    }
    if(flag != 0) {
        fr[count] = seq[i];
        printf("%d\t", fr[count]);
        count++;
        pf++;
    }
    i++;
}
printf("\n");

// After frames are filled, replace pages using the optimal page replacement algorithm
for(i = count; i < max; i++) {
    flag = 1;
    for(j = 0; j < n; j++) {
        if(seq[i] == fr[j]) {

```

```

flag = 0;
break;
}
}
if(flag != 0) {
// Find the page to replace (the one that is used furthest in the future)
for(j = 0; j < n; j++) {
int m = fr[j];
for(k = i + 1; k < max; k++) {
if(seq[k] == m) {
pos[j] = k;
break;
} else {
pos[j] = max + 1;
}
}
}
s = findmax(pos, n); // Find the index of the page to replace
fr[s] = seq[i]; // Replace it with the new page
for(k = 0; k < n; k++) {
printf("%d\t", fr[k]);
}
pf++;
printf("\n");
}
}
// Calculate page fault rate
pfr = (float)pf / (float)max;
printf("\nThe number of page faults are: %d", pf);

```

$$\}$$

Page fault rate: 0.500000

Code:

```
1 #include <stdio.h>
2 int main() {
3     int rs[50], frame[10], n, f;
4     int i, j, k, count = 0, pf = 0;
5     printf("Enter the length of reference string: ");
6     scanf("%d", &n);
7     printf("Enter the reference string: ");
8     for(i = 0; i < n; i++)
9         scanf("%d", &rs[i]);
10    printf("Enter the number of frames: ");
11    scanf("%d", &f);
12    for(i = 0; i < f; i++)
13        frame[i] = -1;
14    printf("\nPage Replacement Process:\n");
15    for(i = 0; i < n; i++) {
16        int found = 0;
17        for(j = 0; j < f; j++) {
18            if(frame[j] == rs[i]) {
19                found = 1;
20                break;
21            }
22        }
23    }
```

```

20         break;
21     }}
22     if(found == 0) {
23         frame[count] = rs[i];
24         count = (count + 1) % f;
25         pf++;
26     }
27     for(k = 0; k < f; k++)
28         printf("%d\t", frame[k]);
29     if(found == 0)
30         printf("PF No. %d", pf);
31     printf("\n");
32 } printf("\nTotal Page Faults = %d", pf);
33 return 0;
34 }

```

Output:

```

Enter the length of reference string: 8
Enter the reference string: 2
3
4
5
0
1
2
3
Enter the number of frames: 4

```

Page Replacement Process:

```

2  -1  -1  -1  PF No. 1
2  3   -1  -1  PF No. 2
2  3   4   -1  PF No. 3
2  3   4   5   PF No. 4
0  3   4   5   PF No. 5
0  1   4   5   PF No. 6
0  1   2   5   PF No. 7
0  1   2   3   PF No. 8

```

Total Page Faults = 8

=== Code Execution Successful ===

Task2: Construct LFU page replacement algorithm by tracking page access frequencies and replacing least frequently used pages.

Code:

```
1  #include <stdio.h>
2  int main() {
3      int rs[50], frame[10], freq[10];
4      int n, f, i, j, k, min, pf = 0;
5      printf("Enter number of page references: ");
6      scanf("%d", &n);
7      printf("Enter reference string: ");
8      for(i = 0; i < n; i++)
9          scanf("%d", &rs[i]);
10     printf("Enter number of frames: ");
11     scanf("%d", &f);
12     for(i = 0; i < f; i++) {
13         frame[i] = -1;
14         freq[i] = 0;
15     }
16     printf("\nPage Replacement Process:\n");
17     for(i = 0; i < n; i++) {
18         int found = 0;
19         for(j = 0; j < f; j++) {
20             if(frame[j] == rs[i]) {
```

```
21                 found = 1;
22                 break;
23             } }
24         if(found == 0) {
25             min = 0;
26             for(k = 1; k < f; k++) {
27                 if(freq[k] < freq[min])
28                     min = k;
29             } frame[min] = rs[i];
30             freq[min] = 1;
31             pf++;
32         } for(j = 0; j < f; j++)
33             printf("%d\t", frame[j]);
34         printf("\n");
35     } printf("\nTotal Page Faults = %d", pf);
36     return 0;
}
```

Output:

```
Enter number of page references: 8
```

```
Enter reference string: 1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
7
```

```
1
```

```
2
```

```
Enter number of frames: 4
```

```
Page Replacement Process:
```

```
1  -1  -1  -1
```

```
1  2   -1  -1
```

```
1  2   3   -1
```

```
1  2   3   4
```

```
5  2   3   4
```

```
7  2   3   4
```

```
1  2   3   4
```

```
5  2   3   4
```

```
7  2   3   4
```

```
1  2   3   4
```

```
1  2   3   4
```

```
Total Page Faults = 7
```

```
=== Code Execution Successful ===
```

Task3: Construct LRU page replacement, replacing the least recently used page when a new page is referenced.

Code:

```
1  #include <stdio.h>
2  int main() {
3      int rs[50], frame[10], time[10];
4      int n, f, i, j, k, least, pf = 0, counter = 0;
5      printf("Enter length of reference string: ");
6      scanf("%d", &n);
7      printf("Enter reference string: ");
8      for(i = 0; i < n; i++)
9          scanf("%d", &rs[i]);
10     printf("Enter number of frames: ");
11     scanf("%d", &f);
12     for(i = 0; i < f; i++) {
13         frame[i] = -1;
14         time[i] = 0;
15     } printf("\nPage Replacement Process:\n");
16     for(i = 0; i < n; i++) {
17         int found = 0;
18         for(j = 0; j < f; j++) {
19             if(frame[j] == rs[i]) {
20                 counter++;
21                 time[j] = counter;
22                 found = 1;
23                 break;
24             }}
25         if(found == 0) {
26             least = 0;
27             for(k = 1; k < f; k++) {
28                 if(time[k] < time[least])
29                     least = k;
30             }
31             counter++;
32             frame[least] = rs[i];
33             time[least] = counter;
34             pf++;
35         } for(j = 0; j < f; j++)
36             printf("%d\t", frame[j]);
37         printf("\n");
38     } printf("\nTotal Page Faults = %d", pf);
39     return 0;
}
```

Output:

```
Enter length of reference string: 7
Enter reference string: 3
4
6
8
1
0
7
Enter number of frames: 4

Page Replacement Process:
3  -1  -1  -1
3  4   -1  -1
3  4   6   -1
3  4   6   8
1  4   6   8
1  0   6   8
1  0   7   8

Total Page Faults = 7
```

Lab Session 14

Objective: Understanding the difference of Indexed, Sequential and Linked File Allocation by implementing them.

Write a C program to simulate the following file allocation strategies.

- Sequential
- Indexed
- Linked

Required Equipment / tools:

- Dev C++

TASK1

DESCRIPTION

A file is a collection of data, usually stored on disk. As a logical entity, a file enables to divide data into meaningful groups. As a physical entity, a file should be considered in terms of its organization. The term "file organization" refers to the way in which data is stored in a file and, consequently, the method(s) by which it can be accessed.

SEQUENTIAL FILE ALLOCATION

In this file organization, the records of the file are stored one after another both physically and logically. That is, record with sequence number 16 is located just after the 15th record. A record of a sequential file can only be accessed by reading all the previous records.

LINKED FILE ALLOCATION

With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block.

INDEXED FILE ALLOCATION

Indexed file allocation strategy brings all the pointers together into one location: an index block. Each file has its own index block, which is an array of disk-block addresses. The *i*th entry in the index block points to the *i*th block of the file. The directory contains the address of the index block. To find and read the *i*th block, the pointer in the *i*th index-block entry is used.

PROGRAM

SEQUENTIAL FILE ALLOCATION

```
#include<stdio.h>

#include<string.h>

struct fileTable {
    char name[20];
    int sb, nob;
} ft[30];

void main() {
    int i, j, n;
    char s[20];

    printf("Enter number of files: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nEnter file name %d: ", i + 1);
        scanf("%s", ft[i].name);

        printf("Enter starting block of file %d: ", i + 1);
        scanf("%d", &ft[i].sb);

        printf("Enter number of blocks in file %d: ", i + 1);
        scanf("%d", &ft[i].nob);
    }

    printf("\nEnter the file name to be searched: ");
    scanf("%s", s);

    for(i = 0; i < n; i++) {
        if(strcmp(s, ft[i].name) == 0)
            break;
    }

    if(i == n)
        printf("\nFile Not Found");
}
```

```

else {
printf("\nFILE NAME\tSTART BLOCK\tNO OF BLOCKS\tBLOCKS OCCUPIED\n");
printf("\n%s\t\t%d\t\t%d", ft[i].name, ft[i].sb, ft[i].nob);
for(j = 0; j < ft[i].nob; j++) {
printf("%d, ", ft[i].sb + j);
}
}
}
}

```

INPUT:

```

Enter no of files      :3
Enter file name 1     :A
Enter starting block of file 1      :85
Enter no of blocks in file 1        :6
Enter file name 2      :B
Enter starting block of file 2      :102
Enter no of blocks in file 2        :4
Enter file name 3      :C
Enter starting block of file 3      :60
Enter no of blocks in file 3        :4
Enter the file name to be searched -- B

```

OUTPUT:

FILE NAME	START BLOCK	NO OF BLOCKS	BLOCKS OCCUPIED
B	102	4	102, 103, 104, 105

LINKED FILE ALLOCATION

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
struct block {

```

```

int bno; // Block number

struct block *next; // Pointer to the next block

};

struct fileTable {

char name[20]; // File name

int nob; // Number of blocks in the file

struct block *sb; // Pointer to the first block

};

void main() {

int i, j, n;

char s[20];

struct block *temp;

printf("Enter number of files: ");

scanf("%d", &n);

// Input file details and allocate blocks for each file

for (i = 0; i < n; i++) {

printf("\nEnter file name %d: ", i + 1);

scanf("%s", ft[i].name);

printf("Enter number of blocks in file %d: ", i + 1);

scanf("%d", &ft[i].nob);

// Allocate memory for the first block

ft[i].sb = (struct block*)malloc(sizeof(struct block));

temp = ft[i].sb;

printf("Enter the blocks of the file: ");

scanf("%d", &temp->bno);

temp->next = NULL;

// Allocate memory for the subsequent blocks and link them

for (j = 1; j < ft[i].nob; j++) {

temp->next = (struct block*)malloc(sizeof(struct block));

```

```

temp = temp->next;
scanf("%d", &temp->bno);
temp->next = NULL;
}
}

// Search for a file by name
printf("\nEnter the file name to be searched: ");
scanf("%s", s);
for (i = 0; i < n; i++) {
    if (strcmp(s, ft[i].name) == 0)
        break;
}
if (i == n) {
    printf("\nFile Not Found");
} else {
    // Output the file's block allocation
    printf("\nFILE NAME NO OF BLOCKS BLOCKS OCCUPIED\n");
    printf("\n%s\t\t%d\t", ft[i].name, ft[i].nob);
    temp = ft[i].sb;
    for (j = 0; j < ft[i].nob; j++) {
        printf("%d -> ", temp->bno);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Free dynamically allocated memory for blocks
for (i = 0; i < n; i++) {
    temp = ft[i].sb;
    while (temp != NULL) {

```

```

struct block *toDelete = temp;

temp = temp->next;

free(toDelete); // Free the block

}

}

}

```

INPUT:

```

Enter no of files : 2

Enter file 1 : A

Enter no of blocks in file
1 : 4

Enter the blocks of the
file 1 : 12 23 9 4

Enter file 2 : G

Enter no of blocks in file
2 : 5

Enter the blocks of the
file 2 : 88 77 66 55 44

Enter the file to be searched :
G

```

OUTPUT:

FILE NAME	NO OF BLOCKS	BLOCKS OCCUPIED
G	5	88 77 66 55 44

INDEXED FILE ALLOCATION

```

#include<stdio.h>

#include<string.h>

struct fileTable {

char name[20]; // File name

int nob; // Number of blocks

int blocks[30]; // Array to store block numbers

```

```

};

void main() {
int i, j, n;
char s[20];

printf("Enter number of files: ");
scanf("%d", &n);

// Input file details and their block allocations
for (i = 0; i < n; i++) {
printf("\nEnter file name %d: ", i + 1);
scanf("%s", ft[i].name);

printf("Enter number of blocks in file %d: ", i + 1);
scanf("%d", &ft[i].nob);

printf("Enter the blocks of the file: ");
for (j = 0; j < ft[i].nob; j++) {
scanf("%d", &ft[i].blocks[j]);
}
}

// Search for a file by name
printf("\nEnter the file name to be searched: ");
scanf("%s", s);

// Search the fileTable array for the given file name
for (i = 0; i < n; i++) {
if (strcmp(s, ft[i].name) == 0) {
break;
}
}

// Output the result of the search
if (i == n) {
printf("\nFile Not Found");
}
}

```

```

} else {
// Print the file details if found
printf("\nFILE NAME NO OF BLOCKS BLOCKS OCCUPIED\n");
printf("%s\t\t%d\t", ft[i].name, ft[i].nob);
for (j = 0; j < ft[i].nob; j++) {
printf("%d, ", ft[i].blocks[j]);
}
printf("\n");
}
}

```

INPUT:

Enter no of files : 2

Enter file 1 : A

Enter no of blocks in file
1 : 4

Enter the blocks of the
file 1 : 12 23 9 4

Enter file 2 : G

Enter no of blocks in file
2 : 5

Enter the blocks of the
file 2 : 88 77 66 55 44

Enter the file to be searched :
G

OUTPUT:

FILE NAME	NO OF BLOCKS	BLOCKS OCCUPIED
G	5	88, 77, 66, 55, 44

Task1: Use file allocation using a linked list to store block numbers for each file.

Code:

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<string.h>
4  struct block {
5      int bno;
6      struct block *next;    };
7  struct fileTable {
8      char name[20];
9      int nob;
10     struct block *sb;
11 } ft[20];
12 int main() {
13     int i, j, n;
14     char s[20];
15     struct block *temp;
16     printf("Enter number of files: ");
17     scanf("%d",&n);
18     for(i=0;i<n;i++) {
19         printf("\nEnter file name %d: ",i+1);
20         scanf("%s",ft[i].name);
21
22         printf("Enter number of blocks in file %d: ",i+1);
23         scanf("%d",&ft[i].nob);
24         ft[i].sb=(struct block*)malloc(sizeof(struct block));
25         temp=ft[i].sb;
26         printf("Enter the blocks of the file: ");
27         scanf("%d",&temp->bno);
28         temp->next=NULL;
29         for(j=1;j<ft[i].nob;j++) {
30             temp->next=(struct block*)malloc(sizeof(struct block));
31             temp=temp->next;
32             scanf("%d",&temp->bno);
33             temp->next=NULL;    }}
34     printf("\nEnter the file name to be searched: ");
35     scanf("%s",s);
36     for(i=0;i<n;i++) {
37         if(strcmp(s,ft[i].name)==0)
38             break;
39     }
40     if(i==n)
41         printf("\nFile Not Found");
42     else {
43         printf("\nFILE NAME\tNO OF BLOCKS\tBLOCKS OCCUPIED\n");
44         printf("%s\t\t%d\t\t",ft[i].name,ft[i].nob);
45         temp=ft[i].sb;
46         while(temp!=NULL) {
47             printf("%d -> ",temp->bno);
48             temp=temp->next;    }
49         printf("NULL");    }
50     return 0; }
```

Output:

```
Enter number of files: 2

Enter file name 1: A
Enter number of blocks in file 1: 4
Enter the blocks of the file: 12
23
8
9

Enter file name 2: G
Enter number of blocks in file 2: 3
Enter the blocks of the file: 23
12
7

Enter the file name to be searched: G

FILE NAME      NO OF BLOCKS      BLOCKS OCCUPIED
G              3          23 -> 12 -> 7 -> NULL
```

```
Enter the file name to be searched: G

FILE NAME      NO OF BLOCKS      BLOCKS OCCUPIED
G              3          23 -> 12 -> 7 -> NULL
```

Task2: Process sequential file allocation, allocating consecutive blocks starting from a given block for each file.

Code:

```
1  #include<stdio.h>
2  #include<string.h>
3  struct fileTable {
4      char name[20];
5      int sb, nob;
6  } ft[20];
7  int main() {
8      int i, j, n;
9      char s[20];
10     printf("Enter number of files: ");
11     scanf("%d",&n);
12     for(i=0;i<n;i++) {
13         printf("\nEnter file name %d: ",i+1);
14         scanf("%s",ft[i].name);
15         printf("Enter starting block of file %d: ",i+1);
16         scanf("%d",&ft[i].sb);
17         printf("Enter number of blocks in file %d: ",i+1);
18         scanf("%d",&ft[i].nob);
19     } printf("\nEnter the file name to be searched: ");
20     scanf("%s",s);
```

```

20     scanf("%s",s);
21     for(i=0;i<n;i++) {
22         if(strcmp(s,ft[i].name)==0)
23             break;
24     }
25     if(i==n)
26         printf("File Not Found");
27     else {
28         printf("\nFILE NAME\tSTART BLOCK\tNO OF BLOCKS\tBLOCKS
                OCCUPIED\n");
29         printf("%s\t\t%d\t\t%d\t\t",
30             ft[i].name,ft[i].sb,ft[i].nob);
31         for(j=0;j<ft[i].nob;j++)
32             printf("%d ",ft[i].sb+j);
33     } return 0;

```

Output:

```

Enter number of files: 2

Enter file name 1: D
Enter starting block of file 1: 34
Enter number of blocks in file 1: 3

Enter file name 2: E
Enter starting block of file 2: 28
Enter number of blocks in file 2: 5

Enter the file name to be searched: D

FILE NAME   START BLOCK NO OF BLOCKS   BLOCKS OCCUPIED
D           34           3           34 35 36

=== Code Execution Successful ===

```

Task3: Construct an index table for each file, storing block numbers where data is allocated.

Code:

```
1  #include<stdio.h>
2  #include<string.h>
3  struct fileTable {
4      char name[20];
5      int nob;
6      int blocks[30];
7  } ft[20];
8  int main() {
9      int i,j,n;
10     char s[20];
11     printf("Enter number of files: ");
12     scanf("%d",&n);
13     for(i=0;i<n;i++) {
14         printf("\nEnter file name %d: ",i+1);
15         scanf("%s",ft[i].name);
16         printf("Enter number of blocks in file %d: ",i+1);
17         scanf("%d",&ft[i].nob);
18         printf("Enter the blocks of the file: ");
19         for(j=0;j<ft[i].nob;j++)
20             scanf("%d",&ft[i].blocks[j]);
21     } printf("\nEnter the file name to be searched: ");
22     scanf("%s",s);
23     for(i=0;i<n;i++) {
24         if(strcmp(s,ft[i].name)==0)
25             break; }
26     if(i==n)
27         printf("File Not Found");
28     else {
29         printf("\nFILE NAME\tNO OF BLOCKS\tBLOCKS OCCUPIED\n");
30         printf("%s\t\t%d\t\t",ft[i].name,ft[i].nob);
31         for(j=0;j<ft[i].nob;j++)
32             printf("%d ",ft[i].blocks[j]);
33     } return 0; }
```

Output:

```
Enter number of files: 2

Enter file name 1: F
Enter number of blocks in file 1: 2
Enter the blocks of the file: 12
89

Enter file name 2: N
Enter number of blocks in file 2: 4
Enter the blocks of the file: 6
8
3
2

Enter the file name to be searched: F

FILE NAME    NO OF BLOCKS    BLOCKS OCCUPIED
F            2              12 89
```

Lab Session 15

Objective: Introduction to kernel development.

Required Equipment / tools:

- **Oracle Virtual Box:** A free and open-source hypervisor for running virtual machines.
- **Fedora:** A popular Linux distribution that provides a powerful and user-friendly environment for users to learn about Linux.

Before starting the lab, ensure the following setup:

Operating System:

- Use a Linux-based OS (Ubuntu, Fedora, or any other Linux distribution).

Development Environment:

- Install GCC (GNU Compiler Collection) for compiling kernel code.
- Install make utility for automation of compilation.
- Install QEMU or use a virtual machine for testing kernel modifications without risking your main OS.

Tools and Libraries:

- Install Kernel Development Headers.
- Install Linux Kernel Source Code for experimentation (can be downloaded from kernel.org).

Building a Simple Kernel:

Configure the Kernel:

- Run the configuration utility to configure the kernel options:
- `make menuconfig`
- Select minimal configurations, such as enabling process management and basic system calls.
- Build the Kernel:

```
make -j$(nproc)
```

Install the Kernel:

```
sudo make modules_install
```

```
sudo make install
```

Task1:

Set up the development environment with VirtualBox, Fedora VM, GCC, make, QEMU, and kernel development headers.

Solution:

The development environment for kernel programming consists of Oracle VirtualBox, which is used to create a virtual machine, and Fedora Linux, which provides a suitable platform for kernel development. GCC is installed to compile the kernel source code, while the make utility automates the compilation process. QEMU or a virtual machine is used to safely test kernel modifications without affecting the host operating system. Kernel development headers and Linux kernel source code are also required for configuring and building a custom kernel.

Task2:

Configure the kernel by enabling minimal features like process management and basic system calls using make menuconfig.

Solution:

The Linux kernel configuration is performed using the make menuconfig utility, which provides a menu-driven interface for selecting kernel features. During configuration, only essential components are enabled to create a lightweight kernel. These components include process management, memory management, loadable module support, and basic system calls. The selected options are saved in a configuration file that is later used during kernel compilation.

Task3:

Build, install, and verify the custom kernel using make, make modules_install, and make install, then check with uname -r.

Solution:

After configuring the kernel, the source code is compiled using the make command. The kernel modules are installed using make modules_install, and the newly built kernel is installed using make install. Once the system is rebooted, the uname -r command is used to verify the kernel version currently running on the system. This confirms that the custom kernel has been successfully built and installed.

Lab Session 16

Assessment of Open-Ended Lab / CCA / PBL				
Blooms Taxonomy	GAs	Knowledge Profile	EAs	WPs
C3 A3	GA-2 GA-6	WK5	EA5	WP5

Title: Dynamic Data Structures: Building a Real-Time Social Media Feed

Motivation:

Operating systems are vital in managing resources and enabling user interactions in computing systems. Linux Fedora, a widely used open-source OS, provides an excellent platform for students to explore real-world system design challenges. This lab encourages students to deepen their understanding of core OS concepts like process management, file systems, and resource allocation while developing practical problem-solving skills.

Concept (Problem Statement):

Students will design and implement a management system using Linux Fedora that addresses a real-world need. The system should utilize core Fedora features and demonstrate the application of operating system principles. Examples of management systems include:

1. System Resource Manager: Monitor and manage system resources (CPU, memory, or disk usage) effectively.
2. Process Management System: Create a tool to handle process operations like monitoring, filtering, and terminating processes.
3. File Management System: Design a system for managing files and directories, including search, backup, and permission settings.
4. Task Automation Manager: Develop a tool for scheduling and automating tasks based on user-defined priorities or time constraints.
5. User Management System: Implement a system for creating, managing, and securing user accounts.

Deliverables

Background/Theory:

Procedure / Methodology:

Data Collection (If required):

Flowchart / Block diagram:

Analysis:

Results:

Discussion on Results:

Concluding Remarks:

Reference:

Open Ended Lab Assessment Rubrics

Course Code and Title: _____

Criteria and Scales				
Excellent (10-8)	Good (7-5)	Average (4-2)	Poor (1-0)	Total Marks 15
Criterion 1: Understanding the Problem: How well the problem statement is understood by the student				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Criterion 2: Research: The amount of research that is used in solving the problem				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Criterion 3: Class Diagram: The completeness of the class diagram				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Criterion 4: Code: How complete and accurate the code is along with the assumptions				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Criterion 5: Report: How thorough and well organized is the solution				
(10-8)	(7-5)%	(4-2)%	(1-0)%	
Total				(____/15)

Total marks: _____

Teacher's signature: _____



Muhammad Soban Khan <mskopian@gmail.com>

List of Topics for Interview

Muhammad Soban Khan <mskopian@gmail.com> Mon, Jul 13, 2026 at 7:49 PM
To: Muhammad Soban Khan <mskopian@gmail.com>, Muhammad Khan <khanofdata@gmail.com>

1. OOP
- All concenpts with examples

Start with this video <https://youtu.be/TfqL31EzqSw>

<https://howtoprogramwithjava.com/object-oriented-programming-java-tutorial/>
2. Data structures
- <http://dotnetslackers.com/Community/files/folders/data-structures-and-algorithms/entry30283.aspx>

<https://www.youtube.com/watch?v=Xzk3XLveA00> (this is just a short video. Search yourself for rest)
3. DB schema and queries
- http://www.tutorialspoint.com/dbms/dbms_overview.htm (check the menu on left hand side of this page for further topics)

<https://www.khanacademy.org/computing/computer-programming/sql>

<http://www.w3schools.com/sql/default.asp>
4. Pseudo code questions
- i.
<https://www.youtube.com/watch?v=rSz7549WSjY>

<https://www.youtube.com/watch?v=Rg-fO7rDsds>

http://www.cosc.canterbury.ac.nz/tim.bell/dt/Tutorial_Pseudocode.pdf

<http://javarevisited.blogspot.com/2011/06/top-programming-interview-questions.html> (don't miss this one. It covers multiple things)
5. Front end questions : Polish your HTML and PHP skills . Don't spend too much time on this one.
6. Frameworks
- <http://www.adfkickstart.com/javajee-software-development-frameworks>

<https://opensource.com/business/15/12/top-5-frameworks>

Spring MVC

Struts

Hiernate
7. Design Patterns
- http://www.tutorialspoint.com/design_pattern/ OR

<http://www.javatpoint.com/design-patterns-in-java>